YourTranscript
A Complete Code Walkthrough

Expanded edition — with line-by-line annotations and exercises. Black & white.

A technical companion for reading the source of the `yt-transcript` project.
Written for a first-year computer-science reader. Estimated reading time: 3.5–5 hours (Parts I & II).

Stack: TypeScript · React 19 · Next.js 16 (App Router) · Tailwind CSS 4 · PostgreSQL (Neon) · Google Gemini

How to read this document

This document explains the entire `yt-transcript` codebase: the folder structure, every source file, the syntax of the languages involved, the reasons behind the main design decisions, the current limits of the application, and what changing or scaling it would require. It assumes you can program a little but does not assume you already know React, Next.js, or web servers.

The material is arranged so you can read it front to back. Earlier chapters introduce vocabulary that later chapters rely on. If a term is unfamiliar, it is either explained where it first appears or listed in the glossary at the end.

Three conventions are used throughout:

- A dark pill like `src/lib/db.ts` names a file in the repository.
- Fixed-width text like `useState` is a piece of code — a function, variable, type, or literal value.
- Indented grey blocks are excerpts of the actual source, quoted verbatim, followed by a line-by-line explanation.

> **On tone.** This is a reference, not a sales pitch. It describes what the code does and why, without adjectives about how good it is. Where a decision has a downside, the downside is stated.

Table of contents

1 · The application in one page

YourTranscript turns a YouTube video into clean, readable text. You paste a link; the app fetches the video's captions, cleans them up with an AI model, and shows you a magazine-style transcript you can read, highlight, annotate, translate, and download.

Around that core, several features have been added:

- **A library.** Every transcript is saved to a database, so it appears in a "Saved Transcripts" list and can be reopened later from any device.
- **A reader with reading settings.** The transcript page has a theme picker (light/dark/sepia), font and size controls, column width, and on-the-fly translation to French or Spanish.
- **Highlights and notes.** You can select text to highlight it (stored in the browser) and attach a note. A dictionary lookup is available for single words or short phrases.
- **Reading-progress memory.** The reader records how far you have scrolled and returns you there next time; the library shows a percentage.
- **A YouTube feed.** If you connect a Google account, the app lists recent videos from your subscriptions, lets you send any of them to the library (which transcribes it), and offers an on-demand AI summary with the video embedded.

The whole thing is one **Next.js** web application. There is no separate backend server written in another language; the "server" parts are functions that live in the same project and run on demand. The database is hosted (Neon PostgreSQL). Three outside services do the heavy lifting the app cannot do itself: **Supadata** (fetches captions from YouTube), **Google Gemini** (the AI that cleans, translates, defines, and summarizes), and the **YouTube Data API** (lists your subscriptions). It is deployed on **Vercel**.

Persistence is split deliberately. Transcripts themselves live in the shared database so they are available anywhere. Everything personal and lightweight — highlights, notes, reading preferences, scroll progress, which feed videos you dismissed — lives in the browser's `localStorage`, tied to the device you are using. There is no login system for end users and no per-user accounts; the app currently behaves as a single-user tool (the Google connection stores one set of tokens).

> **Mental model.** Think of the project as a set of *pages* the user sees, a set of *API endpoints* the pages call, and a small *library of plain functions* the endpoints share. Data comes to rest in two places: a Postgres table (transcripts) and the browser (everything personal).

2 · The languages and tools, and why each is here

Before walking through files, it helps to know what you are looking at and why it was chosen. Five languages/notations appear in this project, plus a handful of frameworks and services.

2.1 · TypeScript and JavaScript

JavaScript is the language web browsers run. Everything interactive on a web page ultimately becomes JavaScript. **TypeScript** is JavaScript with a type system bolted on top: you can annotate what kind of value each variable holds, and a compiler checks those annotations before the code runs. TypeScript does not run in the browser directly — it is *compiled* (translated) to plain JavaScript first. The `.ts` and `.tsx` file extensions mark TypeScript files (the `x` means the file also contains JSX, described under React).

Why TypeScript is used here: the errors it catches are the boring, common ones — passing a number where a string was expected, reading a field that might be missing, typos in property names. In a project with many small pieces talking to each other, that checking removes a whole category of bugs before the code is ever deployed. The cost is that you write more (the type annotations) and occasionally fight the compiler.

A minimal example of the difference:

```
// JavaScript: nothing stops you passing the wrong thing
function greet(name) { return "Hi " + name }

// TypeScript: `name` must be a string; the compiler enforces it
function greet(name: string): string { return "Hi " + name }
```

The `: string` after `name` is a *parameter type*. The `: string` after the parentheses is the *return type*. If you call `greet(42)`, TypeScript refuses to compile. That is the entire idea, applied consistently.

2.2 · React

React is a library for building user interfaces out of *components*. A component is a function that returns a description of what should appear on screen. React's core promise: you describe what the screen should look like *for a given set of data*, and when the data changes, React updates the screen to match. You do not manually poke at the page; you change data and let React re-render.

The description components return is written in **JSX** — HTML-like syntax embedded in the code. This is why files are `.tsx`. A tiny component:

```
function Hello({ name }: { name: string }) {
  return <p>Hello, {name}.</p>
}
```

Reading it: `Hello` is a function taking one argument, an object with a `name` field of type string (that `{ name }: { name: string }` shape is destructuring plus a type, explained in the appendix). It returns JSX: a paragraph containing the text "Hello, " followed by the value of `name`. Curly braces inside JSX switch from "markup" back to "JavaScript expression."

State and hooks. Components remember things using *hooks* — special functions whose names start with `use`. The two you will see constantly:

- `useState` holds a value that can change over time; changing it re-renders the component. `const [count, setCount] = useState(0)` creates a state variable `count` starting at `0` and a function `setCount` to change it.
- `useEffect` runs code *after* the component renders — used for things that reach outside React, like fetching data, reading `localStorage`, or subscribing to browser events. It takes a function and a "dependency array"; it re-runs when any dependency changes.

You will also meet `useRef` (a mutable box that survives re-renders without causing one — often a handle to a DOM element), `useMemo` (compute a value once and reuse it unless inputs change), `useCallback` (the same idea for functions), and `useLayoutEffect` (like `useEffect` but runs before the browser paints, used when you must measure the page). React 19 is the version here.

2.3 · Next.js and the App Router

Next.js is a framework built on top of React. React alone only builds the interface that runs in the browser; it does not, by itself, give you pages with URLs, a server, or a way to run backend code. Next.js supplies all of that. This project uses Next.js 16 with its **App Router**, the newer of its two routing systems.

The App Router's central idea is **routing by folders**. Inside `src/app`, the folder structure *is* the URL structure:

- A file named `page.tsx` in a folder becomes a visitable page at that folder's path. `src/app/library/page.tsx` is the page at `/library`.
- A file named `route.ts` becomes an *API endpoint* — a URL that returns data instead of a page. `src/app/api/transcripts/route.ts` answers requests to `/api/transcripts`.
- A folder named with square brackets, like `[id]`, is a *dynamic segment* — it matches any value and hands it to the code as a parameter. `transcript/[id]/page.tsx` serves `/transcript/abc`, `/transcript/xyz`, and so on, with `id` set accordingly.
- `layout.tsx` wraps every page beneath it with shared structure (here, the `<html>` shell and fonts).

Server and client components. This is the concept that trips up most newcomers. In the App Router, components run on the *server* by default — they render to HTML before being sent to the browser. A component only runs in the browser if its file starts with the line `'use client'`. Browser-only capabilities — `useState`, `useEffect`, click handlers, reading `localStorage`, the `window` object — require a client component. Almost every page in this project starts with `'use client'` because the pages are interactive. The API `route.ts` files are pure server code; they never run in the browser.

> **Why a framework at all?** Without Next.js you would separately set up: a bundler, a router, a Node server for the backend endpoints, and the glue between them. Next.js packages those decisions. The API secrets (database URL, Gemini key) can live safely in server-only code because that code never ships to the browser.

2.4 · Tailwind CSS

CSS is the language that styles web pages — colors, spacing, fonts, layout. Normally you write CSS in separate files with named rules. **Tailwind** is a different approach: instead of writing rules, you attach many tiny pre-made

classes directly to elements. `className="flex items-center gap-2"` means "lay children out in a row, vertically centered, with a small gap." Each of those words is one CSS declaration.

Tailwind version 4 is used here. Two Tailwind conventions appear everywhere:

- **Arbitrary values** in square brackets: `text-[1.837rem]`, `right-[9.3px]`, `bg-[#92908E]`. When no pre-made class fits, you write the exact value in brackets. This project uses many of these because its design is pixel-tuned.
- **Responsive prefixes**: `sm:text-[1.214rem]` means "apply this only on screens at least the `sm` breakpoint wide (640px)." A class with no prefix applies at all sizes; a prefixed one overrides it above the breakpoint. This is how the app adapts to phones.

Custom colors (`bg-purple`, `bg-mint`, `text-trash`) are defined once in `globals.css` and then usable as classes. Chapter 10 covers that file.

2.5 · PostgreSQL and Neon

PostgreSQL ("Postgres") is a relational database: data stored in tables of rows and columns, queried with **SQL**. This project has exactly one table, `transcripts`. **Neon** is a company that hosts Postgres databases in a "serverless" way — you do not run a database server yourself; you connect to Neon's over the network. The project talks to it through the `@neondatabase/serverless` package, which is designed to work from serverless functions (short-lived, on-demand server code) rather than a long-running server.

SQL you will see is small and readable, for example `SELECT * FROM transcripts WHERE video_id = ...`. The library used lets you write SQL inside a special tagged string; it handles safely inserting values so a malicious title cannot become an attack (this is "SQL injection" prevention, discussed with the file).

2.6 · External services

Four outside services are called over the network. Each is reached with an API key or token kept in an environment variable (a named secret configured in the deployment, never written in the code):

Service	What it does here	Secret
Supadata	Given a video ID, returns the raw caption text (the thing YouTube auto-generates).	`SUPADATA_API_KEY`
Google Gemini	The AI model. Cleans raw captions into readable prose; translates; defines words; summarizes.	`GEMINI_API_KEY`
YouTube Data API	Lists the signed-in user's subscriptions and each channel's recent uploads.	OAuth token (per user)
Google OAuth	The sign-in flow that produces the YouTube token.	`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`
Free Dictionary API	Dictionary definitions for single words (no key). Gemini is the fallback.	none

The database connection string (`DATABASE_URL`) is a sixth secret. Keeping all of these out of the code and in environment variables is why the backend logic must run on the server, not in the browser: the browser could not be trusted with them.

3 · The big picture: how a request flows through the system

Before the file-by-file tour, follow one complete action end to end: a user pastes a link and reads the result. Every subsystem appears in this one path.

3.1 · The transcription path, step by step

1. **Home page.** The user types a URL into the box on `/` (the file `src/app/page.tsx`) and clicks "Transcribe." The page does a light check that the text looks like a YouTube link, stores the URL in `sessionStorage` under the key `pending-url`, and navigates to `/transcribing`.
2. **Loading screen.** `src/app/transcribing/page.tsx` reads `pending-url`, shows the animated waves, and makes a network request: `POST /api/transcribe` with the URL in the body.
3. **The transcribe endpoint.** `src/app/api/transcribe/route.ts` runs on the server. It extracts the video ID from the URL, checks the database for a cached transcript, and if none exists, calls Supadata for the raw captions, groups them into paragraphs, sends them to Gemini for cleanup, fetches the title and publish date, saves everything to Postgres, and returns the segments as JSON.
4. **Back on the loading screen.** The response arrives. The page stores the transcript in `localStorage` under `transcript-<id>` so the next page can show it instantly, waits until the animation has run a minimum time, then navigates to `/transcript/<id>`.
5. **The reader.** `src/app/transcript/[id]/page.tsx` reads that `localStorage` copy synchronously so there is no second loading flash, and renders the transcript through the `TranscriptReader` component. From here the reading settings, highlights, translation, and download features take over — all described in their own sections.

Notice the two-place persistence at work: the transcript went into **Postgres** (so it will be in the library forever and on any device) and into **localStorage** (so this device shows it without a round-trip). If you open the same transcript later on a different computer, step 5 will not find a local copy and will instead fetch it from `/api/transcript/<id>`, which reads Postgres.

3.2 · Why a separate loading page at all

Caption fetching plus AI cleanup can take many seconds; a long one can approach the limit. Rather than freeze the home page, the app hands off to a dedicated screen whose only job is to wait and entertain. That screen also enforces a *minimum* visible time so that even a cached, instant response still shows the animation for a consistent moment instead of flickering past. This is a deliberate user-experience decision encoded in constants, covered in the loading-page section.

3.3 · The shape of the data

One data structure recurs everywhere: a transcript is an array of **segments**, and each segment is an object with the text and a start time in milliseconds.

```
interface Segment {
  text: string
  startMs: number
}
```

After cleanup, each segment is one paragraph. The `startMs` is carried through from the caption timings (it is currently set to `0` for cleaned paragraphs, because cleanup regroups the text and the original per-word timings no longer line up — a known simplification). A saved transcript record adds the surrounding metadata:

```
interface TranscriptRecord {
  videoId: string      // the YouTube id, also our primary key
  url: string          // the original link
  title: string        // fetched from YouTube
  author: string       // the channel name
  segments: Segment[]  // the cleaned paragraphs
  createdAt: string    // when we saved it
  publishedAt: string | null  // when the video itself came out
}
```

Hold these two shapes in mind; most files either produce, transform, store, or display them.

4 · The repository, folder by folder

The project is a standard Next.js layout. Everything meaningful lives under `src`. The rest of the top level is configuration.

Top level

Path	Purpose
`package.json`	Lists dependencies and the `dev` / `build` / `start` / `lint` commands.
`package-lock.json`	An exact, machine-generated record of every installed package version. Not edited by hand.
`tsconfig.json`	TypeScript compiler settings.
`next.config.ts`	Next.js settings (essentially empty here).
`postcss.config.mjs`	Wires Tailwind into the CSS build.
`eslint.config.mjs`	Linting rules (style/error checks that run separately from compiling).
`vercel.json`	Deployment settings for Vercel; here, raising one function's time limit.
`next-env.d.ts`	Auto-generated TypeScript glue for Next.js. Never edited.

Inside `src`

Three folders, with a clear division of labor:

Folder	Contains	Runs where
`src/app`	Pages (`page.tsx`), API endpoints (`route.ts`), the root layout, and global CSS. The router reads this folder to build URLs.	Pages: browser. Routes: server.
`src/components`	Reusable pieces of interface shared by pages: the two wave renderers, the reading-settings menu, the transcript reader.	Browser.
`src/lib`	Plain logic with no interface: database access, the AI cleanup call, highlight storage, the Google/YouTube helper, wave geometry math.	Mostly server; a couple are browser utilities.

The full inventory, grouped:

```
src/
  app/
    layout.tsx                 root HTML shell + fonts
    page.tsx                   home page  (/)
    globals.css                all global styles + Tailwind
    icon.svg, favicon.ico      the tab icon
    transcribing/page.tsx      loading screen  (/transcribing)
    transcript/[id]/page.tsx   the reader  (/transcript/:id)
    library/page.tsx           saved transcripts  (/library)
    feed/page.tsx              YouTube feed  (/feed)
    feed/[videoId]/page.tsx    watch + summary  (/feed/:videoId)
    api/
      transcribe/route.ts          POST: make a transcript
      transcript/[id]/route.ts     GET: one transcript from the db
      transcripts/route.ts         GET: list all transcripts
      transcripts/[id]/route.ts    DELETE: remove one
      translate/route.ts           POST: translate a transcript
      define/route.ts              POST: define a word (Gemini fallback)
      summary/route.ts             POST: summarize a video
      feed/route.ts                GET: the user's YouTube feed
      auth/google/route.ts         GET: start Google sign-in
      auth/google/callback/route.ts GET: finish sign-in
      auth/google/status/route.ts  GET: are we connected?
      auth/google/logout/route.ts  GET: disconnect
  components/
    HomeWaves.tsx              the homepage wave animation
    ReadingSettingsMenu.tsx    theme/font/size/width/language menu
    TranscriptReader.tsx       renders text; highlights, notes, dictionary
  lib/
    db.ts                      Postgres access
    cleanup.ts                 raw captions -> clean paragraphs (Gemini)
    highlights.ts              localStorage: highlights, notes, progress
    google.ts                  OAuth + YouTube feed assembly
    wavePath.ts                SVG wave path math
```

This is the whole application: about 3,500 lines. It is small enough to hold in your head once you have read this document.

5 · Configuration files

These files are not glamorous but they decide how everything compiles, styles, and deploys. Read them once and you rarely touch them again.

5.1 · `package.json`

```
{
  "name": "yt-transcript",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint"
  },
  "dependencies": {
    "@neondatabase/serverless": "^1.1.0",
    "next": "16.2.9",
    "react": "19.2.4",
    "react-dom": "19.2.4"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "tailwindcss": "^4",
    "typescript": "^5",
    "eslint": "^9",
    "eslint-config-next": "16.2.9",
    "@types/node": "^20", "@types/react": "^19", "@types/react-dom": "^19"
  }
}
```

Scripts are named shortcuts run with `npm run <name>`. `dev` starts a local development server with live reloading; `build` compiles the production version; `start` serves that build; `lint` runs the style/error checker.

dependencies are packages shipped as part of the running app. There are only four: the Neon database driver, Next.js, and React (the library plus its DOM renderer). That short list is itself a design fact — the project leans on the framework and writes the rest itself rather than pulling in many libraries.

devDependencies are needed only while developing or building, not at runtime: Tailwind and its PostCSS plugin, the TypeScript compiler, ESLint, and the `@types/*` packages (type definitions that teach TypeScript about Node and React). The caret `^` before a version means "this version or any compatible newer one." Exact versions (no caret) pin Next.js and React precisely.

5.2 · `tsconfig.json`

The TypeScript compiler's settings. The lines that matter for reading the code:

- `"strict": true` — turns on all the strict type checks. This is why, for example, a value that might be `null` must be handled before use.
- `"jsx": "react-jsx"` — tells the compiler to understand JSX and how to turn it into React calls.

- `"moduleResolution": "bundler"` and `"module": "esnext"` — use modern `import` / `export` syntax and resolve packages the way a modern bundler does.
- `"noEmit": true` — the compiler only *checks* types; it does not itself produce output files. Next.js does the actual compiling to JavaScript. TypeScript here is purely a safety net.
- `"paths": { "@/*": ["./src/*"] }` — defines the `@/` import shortcut. `import { getTranscript } from '@/lib/db'` means `src/lib/db`, no matter how deep the importing file is. You will see `@/` in nearly every file.

5.3 · `next.config.ts`, `postcss.config.mjs`, `eslint.config.mjs`

`next.config.ts` is effectively empty — the defaults are used. `postcss.config.mjs` registers `@tailwindcss/postcss`, which is the mechanism that turns Tailwind's classes into real CSS at build time. `eslint.config.mjs` pulls in Next.js's recommended lint rule sets (`core-web-vitals` and the TypeScript set) and tells the linter to ignore build output. None of these change how the app behaves; they configure the toolchain around it.

5.4 · `vercel.json`

```
{
  "functions": {
    "src/app/api/transcribe/route.ts": { "maxDuration": 300 }
  }
}
```

Vercel runs each API route as a serverless function with a default time limit (short — often 10 seconds on the free tier). Transcription is the one slow operation: fetching captions plus an AI pass can take a while. This file raises *that one function's* ceiling to 300 seconds (five minutes). The same limit is also declared inside the route file with `export const maxDuration = 300`; the JSON here is the deployment-level counterpart. Every other endpoint keeps the default, which is appropriate because they are fast.

6 · The `lib/` layer — the plain logic

These five files hold logic with no user interface. They are the cleanest place to learn the syntax because there is no JSX in the way. Read this chapter slowly; the patterns here (async functions, `fetch`, error handling, TypeScript interfaces) repeat in every other file.

6.1 · `src/lib/db.ts` — the database

This file is the only place that talks to Postgres. Everything else that needs a transcript goes through the functions it exports. Centralizing database access in one file is a common and deliberate choice: if the storage ever changes, only this file changes.

Setting up the connection

```
import { neon } from '@neondatabase/serverless'

const connectionString = process.env.DATABASE_URL || process.env.POSTGRES_URL
const sql = connectionString ? neon(connectionString) : null

export function dbAvailable(): boolean {
  return sql !== null
}
```

`import { neon } from ...` pulls one function, `neon`, out of the Neon package. `process.env` is how server code reads environment variables; `DATABASE_URL` is the secret connection string. The `||` ("or") means "use `DATABASE_URL`, or if that is missing, `POSTGRES_URL`" — a fallback for different hosting setups.

`const sql = connectionString ? neon(connectionString) : null` uses the **ternary operator** `condition ? a : b`, which is a compact if/else that produces a value: if there is a connection string, build a query function `sql`; otherwise set `sql` to `null`. This is defensive — the app can be built and run in environments without a database, and it simply does nothing storage-related instead of crashing. `dbAvailable()` lets callers check.

Creating the table on first use

```
let ensured = false
async function ensureTable() {
  if (!sql || ensured) return
  await sql`
    CREATE TABLE IF NOT EXISTS transcripts (
      video_id   text PRIMARY KEY,
      url        text NOT NULL,
      title      text,
      author     text,
      segments   jsonb NOT NULL,
      created_at timestamptz NOT NULL DEFAULT now()
    )
  `
  await sql`ALTER TABLE transcripts ADD COLUMN IF NOT EXISTS published_at text`
  ensured = true
}
```

There is no separate database-setup step; the table is created lazily the first time it is needed. `ensured` is a module-level flag so the work happens at most once per running server instance — `if (!sql || ensured) return` bails out early if there is no database or the table was already ensured.

The word `await` appears here for the first time. Database queries take time (they go over the network); an `async` function can `await` such an operation, which means "pause here until this finishes, without freezing everything else." Any function using `await` must be marked `async`. This asynchronous pattern is everywhere in the project because almost everything interesting is a network call.

The SQL itself: `CREATE TABLE IF NOT EXISTS` makes the table only if it is absent. Columns are declared with types — `text`, `jsonb` (JSON stored in the database, used for the segments array), `timestamptz` (a timestamp with time zone). `video_id ... PRIMARY KEY` makes the YouTube ID the unique identifier for each row. The second statement, `ALTER TABLE ... ADD COLUMN IF NOT EXISTS published_at`, is a *migration*: the `published_at` column was added after the table already existed in production, so this line brings older databases up to date without losing data.

The backtick string after `sql` is a **tagged template literal**. The Neon library reads the SQL you write and, crucially, safely substitutes any values you interpolate — you never build SQL by gluing strings together, which is what makes "SQL injection" attacks impossible here.

Reading one transcript

```
export async function getTranscript(videoId: string): Promise<TranscriptRecord | null> {
  if (!sql) return null
  await ensureTable()
  const rows = await sql`SELECT * FROM transcripts WHERE video_id = ${videoId}`
  if (rows.length === 0) return null
  const r = rows[0]
  return {
    videoId: r.video_id,
    url: r.url,
    title: r.title,
    author: r.author,
    segments: r.segments,
    createdAt: String(r.created_at),
    publishedAt: r.published_at ?? null,
  }
}
```

The return type is `Promise<TranscriptRecord | null>`. Two things there: because the function is `async`, it returns a **Promise** — a placeholder for a value that will exist later. And `TranscriptRecord | null` is a **union type**: the result is either a record or nothing. Callers must handle both, which the strict compiler enforces.

The query interpolates `${videoId}` — the Neon tag makes this safe. If no rows come back, the function returns `null`. Otherwise it takes the first row and maps the database's `snake_case` column names (`video_id`) to the code's `camelCase` field names (`videoId`). This mapping is done by hand in one place so the rest of the code deals only in tidy JavaScript names. `r.published_at ?? null` uses the **nullish-coalescing operator** `??`: "use `r.published_at`, unless it is null or undefined, in which case use `null`."

Saving a transcript

```
export async function saveTranscript(rec: {
  videoId: string; url: string; title: string; author: string
  segments: Segment[]; publishedAt?: string | null
}): Promise<void> {
  if (!sql) return
  await ensureTable()
  await sql`
    INSERT INTO transcripts (video_id, url, title, author, segments, published_at)
    VALUES (${rec.videoId}, ${rec.url}, ${rec.title}, ${rec.author},
            ${JSON.stringify(rec.segments)}::jsonb, ${rec.publishedAt ?? null})
    ON CONFLICT (video_id) DO UPDATE
      SET url = EXCLUDED.url, title = EXCLUDED.title, author = EXCLUDED.author,
          segments = EXCLUDED.segments,
          published_at = COALESCE(EXCLUDED.published_at, transcripts.published_at)
  `
}
```

The parameter type is written inline: an object with these fields, where `publishedAt?` has a question mark meaning it is *optional*. Return type `Promise<void>` means the function does its work and resolves with no value.

`JSON.stringify(rec.segments)` converts the segments array into a JSON string, and `::jsonb` casts it into Postgres's JSON type. The `ON CONFLICT (video_id) DO UPDATE` clause is an "upsert": if a row with this video ID already exists, update it instead of failing. That is why re-transcribing a video overwrites cleanly. `COALESCE(EXCLUDED.published_at, transcripts.published_at)` keeps an existing publish date if the new save happens not to have one — it never overwrites good data with nothing.

Delete and list

```
export async function deleteTranscript(videoId: string): Promise<void> {
  if (!sql) return
  await ensureTable()
  await sql`DELETE FROM transcripts WHERE video_id = ${videoId}`
}

export async function listTranscripts(): Promise<TranscriptSummary[]> {
  if (!sql) return []
  await ensureTable()
  const rows = await sql`
    SELECT video_id, url, title, author, created_at, published_at
    FROM transcripts ORDER BY created_at DESC
  `
  return rows.map((r) => ({ /* map columns to a summary object */ }))
}
```

`listTranscripts` selects a lighter set of columns (not the full segments — the library only needs titles and dates) and orders newest-first with `ORDER BY created_at DESC`. `rows.map(...)` transforms every database row into a clean summary object; `.map` is the standard way to turn one array into another by applying a function to each element. The parentheses around the returned object `({ ... })` are required so the arrow function is not mistaken for a function body.

> **What to take from this file.** A single module owns all storage; every function guards against a missing database; column names are translated at the boundary; and query values are always interpolated through

the safe Neon tag. These are the habits worth copying.

6.2 · `src/lib/cleanup.ts` — the AI transcript cleaner

This file turns YouTube's messy auto-captions into readable paragraphs by asking Gemini to rewrite them under strict rules. It is the clearest example in the project of "prompt engineering" — most of the file is an instruction string.

The instruction (system prompt)

A long constant `SYSTEM_PROMPT` tells the model exactly what to do and, just as importantly, what *not* to do. Its rules, in plain terms:

- If the whole transcript arrives in ALL CAPS (YouTube sometimes does this), rewrite it in normal sentence case, capitalizing only real proper nouns, sentence starts, "I", and acronyms.
- Fix misheard proper nouns from context, and apply a fixed list of known mistranscriptions (for example "Palanteer" → "Palantir", "Handes" → "Andes"). This list grows as the user reports new ones.
- Remove the duplicated preview snippet YouTube puts at the very start; remove the censor tokens; thin out filler words by roughly half but not entirely, preserving the speaker's natural rhythm.
- Keep meaningful repetition (emphasis, idioms, laughter) but remove accidental stutters — a nuanced instruction with many examples so the model does not flatten the speech.
- Break the text into paragraphs by speaker turn or by distinct argument. Do **not** prefix paragraphs with speaker names unless the text states the speaker unambiguously and the model is completely certain. Never invent names or use generic labels like "Host."
- Never summarize or drop content; output only the cleaned transcript with blank lines between paragraphs and no markdown.

Each of these rules exists because an earlier version misbehaved in that specific way. The prompt is, in effect, a written record of every correction the project's author has made — which is why it reads like a list of grievances against YouTube's captions.

The call to Gemini

```
const MODEL = 'gemini-2.5-flash'
const ENDPOINT =
`https://generativelanguage.googleapis.com/v1beta/models/${MODEL}:generateContent`

export async function cleanupTranscript(rawText: string): Promise<Segment[] | null> {
  const apiKey = process.env.GEMINI_API_KEY
  if (!apiKey) return null

  let full = ''
  try {
    const res = await fetch(`${ENDPOINT}?key=${apiKey}`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        system_instruction: { parts: [{ text: SYSTEM_PROMPT }] },
        contents: [{ role: 'user', parts: [{ text: rawText }] }],
        generationConfig: { maxOutputTokens: 65536, temperature: 0.2 },
      }),
    })
    if (!res.ok) return null
    const data = await res.json()
    const parts = data?.candidates?.[0]?.content?.parts ?? []
    for (const part of parts) {
      if (typeof part?.text === 'string') full += part.text
    }
  } catch {
    return null
  }
  // ...split into paragraphs...
}
```

`fetch` is the built-in way to make an HTTP request. Here it is a `POST` to Gemini's endpoint, with the API key in the query string, a header declaring the body is JSON, and a body built by `JSON.stringify` from a nested object describing the request. `temperature: 0.2` asks the model to be nearly deterministic — low creativity — because this is a faithful rewrite, not brainstorming. `maxOutputTokens: 65536` allows a long transcript to come back whole.

The `try { ... } catch { ... }` block is error handling: if anything inside `try` throws (network failure, bad JSON), execution jumps to `catch`, which returns `null`. The `?.` in `data?.candidates?.[0]?.content?.parts` is **optional chaining**: read each step only if the previous one exists, otherwise produce `undefined` instead of crashing. This matters because the model's response shape is nested and any level could be missing. The `?? []` then substitutes an empty array so the following loop is safe.

Turning the text back into segments

```
  const paragraphs = full
    .split(/\n\s*\n/)
    .map((p) => p.replace(/\s+/g, ' ').trim())
    .filter(Boolean)

  if (paragraphs.length === 0) return null
  return paragraphs.map((text) => ({ text, startMs: 0 }))
```

The model returns one long string with blank lines between paragraphs. `.split(/\n\s*\n/)` cuts it wherever a blank line appears (that is a **regular expression** matching a newline, optional whitespace, and another newline).

`.map(p => p.replace(/\s+/g, ' ').trim())` collapses any run of whitespace inside a paragraph into single spaces and trims the ends. `.filter(Boolean)` drops empty paragraphs (empty strings are "falsy," so `Boolean` filters them out). Finally each surviving paragraph becomes a `Segment` with `startMs: 0`.

> **Why single-space normalization matters.** The highlight feature stores selections as character offsets into a paragraph's text. Guaranteeing exactly one space between words makes those offsets stable, so a highlight lands on the same characters every time. A small cleanup rule here quietly enables a feature two files away.

The fallback

If Gemini is unavailable or fails, `cleanupTranscript` returns `null`. The transcribe route treats `null` as "use the plain, heuristically grouped segments instead." The app still works without the AI; it just produces rougher text. Designing the AI step to be optional is a deliberate resilience choice.

6.3 · `src/lib/highlights.ts` — highlights, notes, progress

This file is the browser-side counterpart to `db.ts`: it owns everything stored in `localStorage`. Unlike the database module, it runs in the browser, so it guards every access with a check that `window` exists (the same code is prepared on the server during rendering, where `localStorage` is absent).

The Highlight shape and the storage key

```
export interface Highlight {
  id: string
  paragraph: number   // which segment
  start: number       // character offset within the paragraph
  end: number         // exclusive
  text: string        // the highlighted substring
  note: string
  createdAt: number
}

const keyFor = (transcriptId: string, lang: string) =>
`highlights-${transcriptId}-${lang}`
```

A highlight records which paragraph it is in and the `start` / `end` character positions inside that paragraph's text, plus a copy of the highlighted text and an optional note. The storage key includes the **language**: because translating a transcript changes the characters, offsets from the English version would be meaningless in French, so each language keeps its own set of highlights under its own key.

Load and save

```
export function loadHighlights(transcriptId: string, lang: string): Highlight[] {
  if (typeof window === 'undefined') return []
  try {
    const raw = localStorage.getItem(keyFor(transcriptId, lang))
    if (!raw) return []
    const parsed = JSON.parse(raw)
    return Array.isArray(parsed) ? parsed : []
  } catch { return [] }
}

export function saveHighlights(transcriptId: string, lang: string, highlights:
Highlight[]) {
  if (typeof window === 'undefined') return
  try { localStorage.setItem(keyFor(transcriptId, lang), JSON.stringify(highlights)) }
catch {}
}
```

`localStorage` only stores strings, so objects are serialized with `JSON.stringify` on the way in and `JSON.parse` on the way out. Both functions wrap their work in `try/catch` and return a safe default, because `localStorage` can be disabled or full and parsing could fail on corrupted data. `typeof window === 'undefined'` is the standard test for "am I running on the server"; if so, there is nothing to read, so it returns an empty array.

Counting for the library, and clearing on delete

```
export function countHighlights(transcriptId: string): { highlights: number; notes: number
} {
  // scan every localStorage key beginning with `highlights-<id>-`
  // sum total highlights and how many carry a non-empty note
}

export function clearLocalTranscript(transcriptId: string) {
  // remove `transcript-<id>` and every `highlights-<id>-*` key
}
```

`countHighlights` walks all storage keys (using `localStorage.key(i)` in a loop) and adds up highlights and notes across every language for that transcript — this is what produces the two counts shown on each library row. `clearLocalTranscript` is called when a transcript is deleted, so the local cache and its highlights do not linger after the database row is gone.

The offset helper

```
export function charOffsetWithin(root: HTMLElement, node: Node, offset: number): number {
  const walker = document.createTreeWalker(root, NodeFilter.SHOW_TEXT)
  let count = 0
  let n: Node | null = walker.nextNode()
  while (n) {
    if (n === node) return count + offset
    count += n.textContent?.length ?? 0
    n = walker.nextNode()
  }
  return count
}
```

This translates a browser text selection into a character offset. When you select text, the browser reports the selection as a node and an offset within that node — but a paragraph may be split into several text pieces (because a previous highlight wrapped part of it in its own element). A `TreeWalker` visits every text node in order; the function adds up their lengths until it reaches the node the selection started in, then adds the offset. The result is a single number: the position within the paragraph's full text, independent of how the text is currently split into elements. The reader component uses this to store stable highlight ranges.

Reading progress

```
export function loadProgress(transcriptId: string): number { /* read integer 0-100 */ }
export function saveProgress(transcriptId: string, percent: number) { /* clamp and store
*/ }
```

Two small functions store a single integer per transcript: the percentage scrolled. `saveProgress` clamps the value into the 0–100 range and rounds it before storing. The reader writes this on every scroll; the library reads it to show the percentage and the reader reads it to restore your position.

6.4 · `src/lib/google.ts` — OAuth and the YouTube feed

This is the most involved `lib` file. It does two jobs: the OAuth sign-in dance with Google, and assembling a "subscription feed" from the YouTube Data API. It deliberately uses only `fetch` — no Google client library — to avoid an extra dependency.

Is it configured?

```
export function googleConfigured(): boolean {
  return !!(process.env.GOOGLE_CLIENT_ID && process.env.GOOGLE_CLIENT_SECRET)
}
```

`!!(...)` converts a value to a strict `true` / `false`. This returns `true` only if both Google secrets are present. The feed UI uses it to show a "not configured yet" message instead of a broken sign-in when the deployment has no Google credentials.

The three OAuth steps

OAuth is a standard three-step handshake, and this file implements each as a function:

```
export function authUrl(redirectUri: string, state: string): string {
  const params = new URLSearchParams({
    client_id: process.env.GOOGLE_CLIENT_ID ?? '',
    redirect_uri: redirectUri, response_type: 'code',
    scope: 'https://www.googleapis.com/auth/youtube.readonly',
    access_type: 'offline', prompt: 'consent', state,
  })
  return `https://accounts.google.com/o/oauth2/v2/auth?${params}`
}
```

Step one, `authUrl`, builds the Google sign-in link. `URLSearchParams` encodes the query parameters safely. `scope` asks only for read-only access to YouTube. `access_type: 'offline'` is what makes Google return a long-lived *refresh token*; `prompt: 'consent'` forces the consent screen so the refresh token is reliably issued. `state` is a random value used to detect tampering when Google redirects back.

```
export async function exchangeCode(code, redirectUri) { /* POST to /token,
grant_type=authorization_code */ }
export async function refreshAccessToken(refreshToken): Promise<string | null> { /* POST,
grant_type=refresh_token */ }
```

Step two, `exchangeCode`, trades the one-time `code` Google sends back for tokens (an access token and a refresh token). **Step three**, `refreshAccessToken`, uses the long-lived refresh token to get a fresh short-lived access token whenever the feed is loaded. Only the refresh token is stored (in a cookie); access tokens are obtained on demand and never persisted.

Assembling the feed

YouTube offers no single "my subscription feed" endpoint, so the feed is built by hand. `getFeed(token)` performs a sequence of calls, each through a small helper `yt(path, token)` that attaches the access token and parses the JSON:

1. List the user's subscriptions (paginating up to about 100 channels).
2. For each channel, look up its "uploads" playlist ID (channels are queried 50 at a time, the API's batch limit).
3. For each uploads playlist, fetch the recent items, keeping only those published within the last 30 days.
4. Collect the candidate video IDs and fetch their durations in batches of 50.
5. Drop "Shorts" — videos 180 seconds or shorter, or with "#shorts" in the title — then sort everything newest-first.

```
const isShort = (v: FeedVideo) => {
  const dur = durationById[v.videoId] ?? 0
  return (dur > 0 && dur <= 180) || /#shorts?\b/i.test(v.title)
}
return candidates.filter((v) => !isShort(v))
  .sort((a, b) => +new Date(b.publishedAt) - +new Date(a.publishedAt))
```

The date parsing `+new Date(...)` turns a date string into a number of milliseconds so two dates can be subtracted; sorting by `b - a` yields descending (newest first) order. `isShort` combines a duration test with a title pattern. The 30-day window, the batching, and the caps all exist to keep within YouTube's daily API quota — a constraint discussed in the scaling chapter.

Several calls run in parallel with `await Promise.all(uploads.map(async (u) => { ... }))`. `Promise.all` starts many asynchronous operations at once and waits for all of them, rather than doing them one at a time — important when there are dozens of channels to query.

6.5 · `src/lib/wavePath.ts` — the decorative wave geometry

This file is pure math and produces no data or network calls: it generates the **SVG path** strings that draw the calligraphic "waves" on the home and loading pages. SVG is a vector-graphics format where a shape is described by a string of drawing commands.

```
export function makeWaveOutline(seed: number, maxHalfWidth: number) {
  const cycles = 1
  const amp = 4.8 + (seed % 3)   // amplitude varies slightly per wave
  const mid = VIEW_H / 2
  const N = 48
  const pts = []
  for (let i = 0; i <= N; i++) {
    const t = i / N
    pts.push({ x: t * VIEW_W, y: mid + amp * Math.sin(2 * Math.PI * cycles * t) })
  }
  // ...offset a tapered ribbon above and below the centerline...
}
```

The function samples a sine wave at 48 points along a fixed viewbox to get a gently curved centerline (`Math.sin` produces the up-and-down shape; `amp` is how tall, varied per `seed` so no two waves are identical). It then walks that centerline and pushes each point outward by a half-width that is zero at the ends and largest in the middle (`maxHalfWidth * Math.sin(Math.PI * t)`), producing a filled ribbon that tapers to points — the brush-stroke look. The result is a single `d="..."` string of `M` (move) and `L` (line) commands closed with `Z`.

The file also exports `WAVE_SHADOWS` (a lookup from each wave color to a lighter version used as its drop shadow) and constants for the viewbox size and a pixels-per-centimeter factor. The two page components import these and animate many copies with CSS. This is the one corner of the project that is closer to graphics programming than to web plumbing, and it is fully self-contained — nothing outside the two wave components depends on it.

7 · The API routes — the server endpoints

Every file named `route.ts` under `src/app/api` is a server endpoint. Pages in the browser call these with `fetch`; the endpoints do the work that needs secrets or the database, then return JSON. A route file exports functions named after HTTP methods — `GET`, `POST`, `DELETE` — and Next.js calls the matching one.

7.1 · `api/transcribe/route.ts` — the centerpiece

This is the busiest endpoint and the heart of the app. It accepts a URL and returns a finished transcript. Read it as five stages.

Stage 1 — validate and extract the ID

```
export const maxDuration = 300

function extractVideoId(url: string): string | null {
  try {
    const u = new URL(url)
    if (u.hostname === 'youtu.be') return u.pathname.slice(1).split('?')[0]
    return u.searchParams.get('v')
  } catch { return null }
}

export async function POST(req: Request) {
  const { url } = await req.json()
  if (!url) return NextResponse.json({ error: 'No URL provided' }, { status: 400 })
  const videoId = extractVideoId(url)
  if (!videoId) return NextResponse.json({ error: 'Invalid YouTube URL.' }, { status: 400
})
```

`export const maxDuration = 300` is the in-code twin of the `vercel.json` setting: this function may run up to five minutes. `extractVideoId` handles both YouTube URL shapes — the `youtu.be/ID` short form and the `youtube.com/watch?v=ID` long form — using the built-in `URL` parser rather than fragile string cutting. `const { url } = await req.json()` reads and destructures the request body. Failures return a JSON error with an HTTP **status code**: `400` means "bad request." Status codes are how the caller knows success (200-range) from failure (400- and 500-range).

Stage 2 — the cache

```
const cached = await getTranscript(videoId)
if (cached) {
  return NextResponse.json({ id: videoId, segments: cached.segments, title: cached.title,
cached: true })
}
```

Before spending money on Supadata and Gemini, the route checks the database. If this video was transcribed before, it returns the stored version immediately. This is the single most important cost- and speed-saving decision in the app: transcription happens once per video, ever, across all users.

Stage 3 — fetch the captions

```
const res = await fetch(
  `https://api.supadata.ai/v1/youtube/transcript?videoId=${videoId}&lang=en`,
  { headers: { 'x-api-key': apiKey } }
)
if (!res.ok) { /* return a 422 "no transcript available" with the reason */ }
const data = await res.json()
raw = data.content ?? []
```

Supadata is asked for the English captions. If it fails, the route returns status `422` ("unprocessable") with a human-readable reason — this is what surfaces as "No transcript available for this video" when captions are disabled. The raw result is an array of caption fragments, each with text, an offset, and a duration.

Stage 4 — group into rough paragraphs

Before the AI sees the text, the route does a first, mechanical grouping. Two strategies:

- If the captions contain `>>` markers (YouTube's convention for a new speaker), a new block starts at each marker.
- Otherwise, blocks are split on time gaps: when more than two seconds pass between fragments, a new paragraph begins.

A second pass then merges fragments that begin with a lowercase letter into the previous block, since a lowercase start usually means a continued sentence rather than a new speaker. This heuristic grouping is a safety net: if the AI step is skipped or fails, the transcript is still broken into reasonable paragraphs.

Stage 5 — clean, look up metadata, save, return

```
const rawText = raw.map((item) => item.text).join(' ')
const cleaned = await cleanupTranscript(rawText)
const finalSegments = cleaned ?? merged

const [{ title, author }, publishedAt] = await Promise.all([
  fetchTitle(videoId), fetchPublishDate(videoId),
])
await saveTranscript({ videoId, url, title, author, segments: finalSegments, publishedAt
})
return NextResponse.json({ id: videoId, segments: finalSegments, title })
```

The full caption text is joined into one string and passed to `cleanupTranscript` (the file from 6.2). `const finalSegments = cleaned ?? merged` is the fallback in action: use the AI-cleaned paragraphs, or the mechanically grouped ones if cleanup returned `null`. Title and channel come from YouTube's keyless **oEmbed** endpoint; the publish date is scraped from the public watch page (there is no keyless API for it, so `fetchPublishDate` pulls it out of the page's HTML with a regular expression). Both metadata calls run together via `Promise.all`. Finally the record is saved and the segments are returned to the loading page.

> **Reading this route teaches the whole backend style.** Validate input; check the cache; call an external service with a key; transform; store; return JSON with a status code. Every other endpoint is a smaller version of this pattern.

7.2 · The transcript CRUD routes

Three tiny endpoints wrap the database functions. "CRUD" is create/read/update/delete; these cover read, list, and delete (create happens inside `transcribe`).

- `api/transcript/[id]/route.ts` — `GET`. Returns one transcript by ID from the database (used when opening a transcript on a device that has no local copy). Note the dynamic segment: `{ params }: { params: Promise<{ id: string }> }` and `const { id } = await params` — in this Next.js version the route parameters arrive as a Promise and must be awaited.
- `api/transcripts/route.ts` — `GET`. Returns the whole list for the library. Four lines.
- `api/transcripts/[id]/route.ts` — `DELETE`. Removes one transcript. The first parameter is named `_req` with a leading underscore, a convention meaning "required by the signature but unused."

7.3 · `api/translate/route.ts`

Given a title and paragraphs and a target language (French or Spanish), this asks Gemini to translate them. The interesting part is how it keeps paragraphs aligned. It joins all chunks with a rare delimiter, `\n@@@\n`, and instructs the model to keep exactly the same number of chunks separated by that same delimiter. On the way back it splits on the delimiter and checks the count:

```
if (out.length !== chunks.length) {
  return NextResponse.json({ error: 'Translation misaligned.' }, { status: 502 })
}
```

If the counts do not match, it fails rather than returning misaligned text — because a paragraph mismatch would put highlights on the wrong words. This is a small example of a recurring principle: when correctness cannot be guaranteed, fail loudly instead of returning something subtly wrong.

7.4 · `api/define/route.ts` and `api/summary/route.ts`

define is the dictionary fallback. The reader first tries the free, keyless dictionary API directly from the browser; only when that returns nothing (common for non-English words and for phrases) does it call this route, which asks Gemini for a one-line definition in the target language, formatted as `part-of-speech ||| definition` and split apart on the client.

summary powers the "Ask AI" button on the feed watch page. It gathers the video's caption text — reusing a saved transcript if one exists, otherwise fetching fresh from Supadata — and asks Gemini for a one-line TL;DR followed by five to eight bullet points. If the video has no captions, it returns an empty summary and the page says so. Both routes follow the same call-Gemini-and-parse structure as `cleanup`.

7.5 · `api/feed/route.ts` and the auth routes

feed reads the refresh token from the request cookie, exchanges it for a fresh access token via `refreshAccessToken`, calls `getFeed`, and returns the list. No cookie means status `401` ("unauthorized"), which the feed page reads as "not connected."

The four **auth/google** routes implement the sign-in flow described in 6.4:

- `route.ts` (`GET`) — generates a random `state`, stores it in a short-lived cookie, and redirects the browser to Google's consent screen.
- `callback/route.ts` (`GET`) — Google returns here with a code; the route checks the `state` matches (anti-tampering), exchanges the code for tokens, stores the refresh token in an `httpOnly` cookie (unreadable by JavaScript, so it cannot be stolen by a script), and redirects to `/feed`.
- `status/route.ts` (`GET`) — reports whether Google is configured and whether a token cookie is present, so the page can pick which state to show.
- `logout/route.ts` (`GET`) — deletes the token cookie.

Storing the token server-side in an `httpOnly` cookie rather than in `localStorage` is a security decision: page scripts never see it, which limits the damage a malicious script could do. It is also why the feed is currently single-user — there is one cookie per browser, not a per-account token store.

8 · The React components

Three components in `src/components` are shared by pages. The first two are self-contained; the third, the transcript reader, is the most intricate client code in the project and rewards careful reading.

8.1 · `HomeWaves.tsx`

A client component (`'use client'`) that renders the animated background on the home page. It builds a list of wave descriptions once with `useMemo` — six rows, fourteen per row, each with a pseudo-random size, vertical position, animation delay and duration derived from a running `seed` number — then filters out every fourth one to thin the field evenly. It maps that list to `<svg>` elements, each drawing one wave via `makeWaveOutline` and animating it horizontally with a CSS animation named `wave-drift`. The negative `delay` values start each wave partway through its cycle so the screen is already full of motion on load rather than starting empty.

```
const waves = useMemo(() => {
  const items = []
  let seed = 0
  for (let row = 0; row < ROWS; row++)
    for (let i = 0; i < WAVES_PER_ROW; i++) {
      seed++
      const duration = (14 + ((seed * 1.1) % 10)) * 1.3 * SPEED_FACTOR
      items.push({ seed, topPercent: /* row position + jitter */, sizeCm: /* varied */,
                   delay: -(((seed - 1) / total) * duration), duration,
                   color: WAVE_COLORS[(seed * 7) % WAVE_COLORS.length] })
    }
  return items.filter((_, idx) => (idx + 1) % 4 !== 0)
}, [])
```

`useMemo(() => {...}, [])` computes the array a single time (the empty dependency array `[]` means "never recompute"). The arithmetic with `seed` and the modulo operator `%` is a cheap way to get varied-but-fixed values without a real random-number generator, so the layout is stable between renders. Every tuning constant — `SPEED_FACTOR`, `HALF_STROKE_WIDTH`, the size range — is named at the top of the file, a sign the visuals were adjusted many times.

8.2 · `ReadingSettingsMenu.tsx`

This file has two responsibilities: it defines the vocabulary of reading options (themes, fonts, size/spacing/width steps, languages) as exported constants, and it renders the pop-over menu that changes them. It also exports a custom hook, `useReadingPrefs`, that other files use to read and persist the settings.

The option catalogs

```
export const THEMES = {
  white: { label: 'White', bg: '#FFFFFF', text: '#1A1A1A', shadow: 'rgba(0,0,0,0.15)' },
  sepia: { label: 'Sepia', bg: '#F4ECD8', text: '#4A3728', shadow: 'rgba(0,0,0,0.15)' },
  /* paper, cream, dark, carbon, shaded, forest ... */
} as const

export const SIZE_STEPS = [0.875, 1, 1.125, 1.25, 1.375, 1.5]
export const WIDTH_STEPS = ['44rem', '50rem', '56rem', '62rem', '68rem']
```

Each theme is an object with a background, text color, and shadow tint. `as const` tells TypeScript to treat these as fixed literal values, which makes the keys (`'white' | 'sepia' | ...`) into a precise type used elsewhere. The steps are plain arrays; the menu stores an *index* into them, not the value, so the controls are simple increment/decrement steppers.

The preferences hook

```
export function useReadingPrefs() {
  const [prefs, setPrefs] = useState<ReadingPrefs>(DEFAULT_PREFS)
  const [loaded, setLoaded] = useState(false)

  useEffect(() => {   // on mount: read saved prefs, sanitize, apply
    const raw = localStorage.getItem(STORAGE_KEY)
    if (raw) setPrefs((p) => {
      const merged = { ...p, ...JSON.parse(raw) }
      if (!(merged.font in FONTS)) merged.font = DEFAULT_PREFS.font
      /* ...same guard for theme, lang, and each index... */
      return merged
    })
    setLoaded(true)
  }, [])

  useEffect(() => { if (loaded) localStorage.setItem(STORAGE_KEY, JSON.stringify(prefs))
}, [prefs, loaded])
  return [prefs, setPrefs] as const
}
```

This is a **custom hook** — a function that uses other hooks and packages a reusable behavior. It holds the preferences in state, loads any saved version once on mount, and saves them again whenever they change. The `merged` block is defensive: an old saved setting might name a font or theme that has since been removed, so each field is validated against the current catalogs and reset to the default if unrecognized. This is a real bug fix encoded permanently — an unrecognized font once caused the reader to crash. The `loaded` flag prevents the save effect from firing before the initial load, which would overwrite saved prefs with defaults.

The menu, steppers, and dropdown

The default export is the menu component. It closes itself when you click outside or press Escape (a `useEffect` that adds and later removes document-level event listeners — the cleanup function returned from the effect is what removes them). Inside are a theme swatch grid, a custom `FontDropdown`, three `Stepper` controls (font size, spacing, width), and a language selector. The `Stepper` and `FontDropdown` are small local components defined in the same file because they are used nowhere else. The whole menu is scaled to 1.1× from its top-right corner so it grows leftward while staying pinned to the button — a CSS `transform` detail that keeps the layout from shifting.

8.3 · `TranscriptReader.tsx` — the reading surface

At around 500 lines this is the largest component. It renders the paragraphs and layers three interactive features on top: highlighting, notes, and dictionary lookup. It is worth understanding because it demonstrates how far a single well-structured client component can go.

State

```
const [highlights, setHighlights] = useState<Highlight[]>([])
const [popup, setPopup] = useState<Popup>(null)
const containerRef = useRef<HTMLDivElement>(null)
```

`highlights` is the list of stored highlights for the current transcript and language, loaded from `localStorage` on mount and saved on every change through a `persist` helper. `popup` is a single piece of state describing which floating UI is currently open — a **discriminated union** type: it is either `null`, or an object whose `kind` field is `'actions'` (the Highlight/Dictionary buttons after selecting text), `'note'` (the note editor), or `'define'` (the dictionary result). Storing all pop-ups in one variable with a `kind` tag means only one can be open at a time and the render logic can switch on `kind`.

Turning a selection into a stored range

```
const handleMouseUp = useCallback(() => {
  setTimeout(() => {
    const sel = window.getSelection()
    if (!sel || sel.isCollapsed) return
    const range = sel.getRangeAt(0)
    // find the paragraph elements the selection starts and ends in;
    // for each paragraph it touches, record start/end character offsets
    // using charOffsetWithin(); build a list of per-paragraph "spans"
  }, 0)
}, [])
```

When the mouse is released, the component reads the browser's current text selection. Because a selection can cross paragraph boundaries, it walks every paragraph the selection touches and, using `charOffsetWithin` from `highlights.ts`, converts the visual selection into one or more `{paragraph, start, end}` spans. Those spans are what get stored, so a highlight survives re-rendering and re-opening. The `setTimeout(..., 0)` defers the work by one tick so that any click on an existing highlight is handled first.

Merging overlapping highlights

When you highlight text that overlaps an existing highlight, the two are merged into one rather than stacked. A helper `mergeSpan` finds highlights in the same paragraph whose ranges overlap the new one, computes the union of their start/end, keeps any notes, and replaces them all with a single merged highlight. This keeps the stored data clean and the rendering unambiguous.

Rendering highlighted text

To display a paragraph with highlights, the component cannot just print the text — it must wrap the highlighted character ranges in their own elements. It collects all the highlight boundaries in a paragraph, sorts them into a set of break points, and walks left to right emitting either a plain text run or a `<mark>` element for each slice between break points. Speaker-name prefixes (text before the first colon) are rendered bold-italic. This slice-and-wrap approach is the standard way to render styled ranges over plain text.

The two chronological sections

Below the transcript, the component renders an "All Highlights" section and a "Notes" section, each listing entries in creation order. The notes section shows only highlights that carry a note. Both sections are given HTML anchors (`id="highlights"` , `id="notes"`) so the library's count badges can link directly to them; an effect scrolls to the requested section when the page opens with that anchor in the URL.

Why this lives in one component. Highlighting, notes, and dictionary all operate on the same selection and the same text, and all render pop-ups positioned over that text. Keeping them together avoids passing selection state between components. The cost is a large file; the benefit is that the interactions never fight each other.

9 · The pages

Each `page.tsx` is one screen the user visits. All are client components because all are interactive. They share a visual language — the same header with the underlined wordmark and mint buttons — but each owns its own logic.

9.1 · `layout.tsx` and fonts

The root layout wraps every page. It is one of the few *server* components (no `'use client'`). It loads four fonts through `next/font/google` — Space Grotesk (headline), Anton (display), Source Serif 4 (serif body), Inter (sans body) — and exposes each as a CSS variable on the `<html>` element. Those variables (`--font-headline-family`, etc.) are what the Tailwind font classes point at. It also exports `metadata` (the page title and description used by browsers and link previews) and a `viewport` setting (`width=device-width, initial-scale=1`) that makes the layout render at true size on phones.

```
export const viewport: Viewport = { width: 'device-width', initialScale: 1 }

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en" className={`${spaceGrotesk.variable} ${sourceSerif.variable} ...`}>
      <body>{children}</body>
    </html>
  )
}
```

`children` is whatever page is being shown; the layout simply places it inside the shared HTML shell. Loading fonts here means they are declared once and available to all pages through the CSS variables.

9.2 · `page.tsx` — the home page

The landing screen: header, a headline, the URL input, and the wave section. Its only real logic is submitting the URL. On submit it does a light validity check (does the text contain `youtube.com` or `youtu.be`), stashes the URL in `sessionStorage`, and navigates to `/transcribing` using the router from `next/navigation`. It also contains a small piece of measurement code: a `useEffect` that, on desktop only, stretches the input row so its right edge lines up with the end of the headline — a purely cosmetic alignment that is disabled below the phone breakpoint to avoid overflow.

```
const handleSubmit = () => {
  if (!url.trim()) { setError('Put a YouTube URL'); return }
  if (!isValidYoutubeUrl(url)) { setError("It doesn't look like a youtube URL"); return }
  setLoading(true)
  sessionStorage.setItem('pending-url', url)
  router.push('/transcribing')
}
```

`sessionStorage` (like `localStorage` but cleared when the tab closes) is used to hand the URL to the next page without putting it in the visible address bar. This is a small, deliberate choice: the loading page has no other way to know which URL to transcribe.

9.3 · `transcribing/page.tsx` — the loading screen

Covered in outline in Chapter 3. The mechanics: it builds a field of waves as mirrored left/right pairs that reveal bottom-up, driven by named constants at the top of the file. It reads `pending-url`, calls `/api/transcribe`, and on success stores the result in `localStorage` and navigates to the reader. The notable engineering detail is the **minimum visible time**:

```
const FILL_COMPLETE_S = (WAVE_ROWS - 1) * ROW_APPEAR_STEP + 0.4
const MIN_VISIBLE_MS = Math.ceil((FILL_COMPLETE_S + 0.6) * 1000)
// ...
const waitForMinimum = async () => {
  const elapsed = Date.now() - startedAt
  if (elapsed < MIN_VISIBLE_MS) await new Promise((r) => setTimeout(r, MIN_VISIBLE_MS -
elapsed))
}
```

The minimum is computed from the animation constants, not guessed. Even when the transcript comes back instantly from cache, the page waits so the animation completes at least once — every visit sees the same unhurried fill instead of a flash. A guard ref (`startedRef`) ensures the fetch runs only once even though effects can re-run. On error it shows a message and returns to the home page after a short delay.

9.4 · `transcript/[id]/page.tsx` — the reader

The richest page. It coordinates the reading settings, translation, download, scroll-progress memory, the title's per-line underline, and delegates the actual text to `TranscriptReader`. Key pieces:

- **Instant load.** It reads the local copy synchronously in the initial state (`useState(() => readLocalTranscript(id))`) so the common path — arriving from the loading page — never flashes a spinner. In parallel a `useEffect` tries the database, so a direct link on a fresh device also works.
- **Reading preferences.** It uses `useReadingPrefs` and applies the chosen theme, font, size, spacing, and width to the reading container through inline styles. Dark themes get special treatment — the title underline and word shadows are tuned per surface so they read well on black.
- **Translation.** When the language is not English, it calls `/api/translate`, caches the result in memory and `localStorage`, and shows the translated title and paragraphs. Switching back is instant because both versions are cached.
- **Scroll progress.** A scroll listener writes the percentage to `localStorage`; a one-time effect restores the position on open unless the URL points at a specific section.
- **Download.** A "Download" menu offers plain text (built by joining the paragraphs) and PDF (which triggers the browser's own print-to-PDF against a hidden two-column layout that uses the chosen reading font).
- **Delete.** A button at the end removes the transcript from the database and clears the local copy, then returns to the library.

The per-line title underline is a good example of doing something the platform cannot do directly: CSS underlines cannot have a separate shadow color, so the page measures each wrapped line of the title with `Range.getClientRects()` and draws its own purple bar under each line. This measure-and-draw pattern (used again in the library's hover underline) is how the app achieves effects beyond plain CSS.

9.5 · `library/page.tsx`

Fetches the transcript list from `/api/transcripts` and renders it as boxed rows. For each row it computes, from `localStorage`, the highlight and note counts and the reading percentage, and shows them on the right. Each row's title uses the measure-and-draw underline on hover; a shelf-less trash button deletes the transcript (database plus local cache); the count badges link into the reader's highlight/notes sections. A small local component, `RowTitle`, measures the title's wrapped lines so the hover underline hugs the last line rather than spanning the full width.

9.6 · `feed/page.tsx` and `feed/[videoId]/page.tsx`

feed/page.tsx first calls `/api/auth/google/status` and branches: not configured, not connected (show a Connect button linking to the OAuth start route), or connected (fetch `/api/feed` and list the videos). Each video row has a shelf button that sends the video to the library by calling `/api/transcribe` and then removes it from the feed, and a trash button that dismisses it (remembered in `localStorage`). Dismissed videos are filtered out of the list.

feed/[videoId]/page.tsx is the watch page: it embeds the YouTube player in an `<iframe>` inside a thick purple bordered box, offers an "Ask AI" button that calls `/api/summary` on demand and renders the TL;DR and bullets, and has a delete button that dismisses the video. The summary is not fetched automatically — only when asked — which keeps the page cheap to open.

10 · Styling: `globals.css` and how Tailwind is wired

One CSS file, `src/app/globals.css`, defines the design tokens, a few global rules, the custom scrollbar, and the wave-animation keyframes. Everything else is Tailwind classes on elements.

```
@import "tailwindcss";

@theme {
  --color-purple: #4E00FF;
  --color-mint:   #54FFC9;
  --color-paper:  #ECEAE8;
  --color-trash:  #F5402E;   /* the one deliberate step outside the palette */
  --font-headline: var(--font-headline-family);
  /* ...more colors and font links... */
}

body { @apply bg-cream text-ink font-body antialiased; }
```

`@import "tailwindcss"` pulls in the whole framework. The `@theme` block is Tailwind 4's way of declaring design tokens: each `--color-*` variable automatically becomes usable as a class — `--color-purple` makes `bg-purple`, `text-purple`, `border-purple` all work. The font variables point at the ones the layout defined from the loaded fonts, connecting `font-headline` to Space Grotesk. `@apply` lets a normal CSS rule reuse Tailwind classes, used here to style the `body`.

The rest of the file is a handful of things Tailwind classes cannot express: the global focus outline color, an override so the reading menu shows no focus ring, a custom thin black scrollbar for the font dropdown (with every arrow-button variant hidden, a fix that took several tries across browsers), the print settings, and the `@keyframes` definitions that animate the waves horizontally. Keeping these few rules in one place — and everything else as inline classes — is the Tailwind philosophy: styling lives next to the markup, and the global file holds only what must be global.

11 · Cross-cutting concepts, in depth

The first edition listed the recurring ideas. This edition explains each one far enough that you could defend it in a code review. Read this chapter as the conceptual spine; every later annotation refers back to it.

11.1 · Asynchrony: promises, `await`, and why the UI never freezes

A web page runs on a **single thread**. There is one line of execution; if it is busy, nothing else happens — clicks are ignored, animations stall. A network request takes tens or hundreds of milliseconds. If the code simply "waited" on that thread, the page would freeze for the duration. JavaScript avoids this with an **event loop**: slow work is handed off, the thread continues, and when the work finishes a callback is queued to run.

A **Promise** is the object that represents "a value that is not ready yet." It is in one of three states: pending, fulfilled (with a value), or rejected (with an error). You can attach reactions with `.then()` and `.catch()`, but the readable form is `async/await`: inside an `async` function, `await somePromise` pauses *that function* until the promise settles, while the thread is free to do other work. The pause is cooperative, not a freeze.

```
// These two are equivalent; the second is how the project writes it.
fetch(url).then(r => r.json()).then(data => use(data))
const r = await fetch(url); const data = await r.json(); use(data)
```

Three consequences you will see enforced throughout the code:

- **Contagion.** If a function `await`s, it must be `async`, and its callers must `await` it (or handle the promise). Asynchrony spreads up the call chain. That is why the API routes are all `async function POST(...)`.
- **Errors travel differently.** A rejected promise inside `await` throws, so it is caught by `try/catch`. A rejection you forget to await becomes an "unhandled rejection." The project wraps every external call in `try/catch` for exactly this reason.
- **Parallel vs. serial.** `await a(); await b();` runs them one after another. `await Promise.all([a(), b()])` starts both immediately and waits for both. `google.ts` and the transcribe route use `Promise.all` to fetch many things at once; doing them serially would multiply the wait.

11.2 · The server/client boundary, concretely

In the App Router a component is a **server component** unless its file's first line is `'use client'`. This is not a stylistic tag; it changes where the code physically runs and what it may touch.

	Server component / route	Client component
Runs	On the server, before the response is sent	In the browser, after the HTML arrives
May read secrets (`process.env`)	Yes	No — the browser must never see them
May use the database	Yes	No
May use `useState`, `useEffect`, event handlers	No	Yes

May read `localStorage`, `window`	No (they do not exist there)	Yes

The mental test: does this code need a secret or the database? It must be server. Does it need interactivity or browser storage? It must be client. In this project the split is stark — the `route.ts` files and `layout.tsx` are server; every `page.tsx` and every component are client. The `lib` files are imported by whichever side needs them: `db.ts` and `google.ts` only by server routes; `highlights.ts` only by client code (hence its `typeof window === 'undefined'` guards, which make it harmless if it is ever pulled onto the server during rendering).

11.3 · Two-place persistence, and the consistency price

Durable, shareable data (the transcripts themselves) lives in Postgres. Personal, per-device data (highlights, notes, preferences, scroll progress, feed dismissals) lives in the browser's `localStorage`. This is not an accident of laziness; it is a deliberate trade that buys "no login required" at the cost of "personal data does not follow you."

Understand precisely what each store guarantees. Postgres is one source of truth for all visitors: transcribe a video on one machine and it is in the library on every machine. `localStorage` is a per-origin, per-browser key–value store of strings; it is invisible to other devices and to other browsers on the same device, and clearing site data erases it. The project leans into this: the library's highlight counts and reading percentages are computed from `localStorage` on whatever machine you are on, which is why the first edition warned they are device-bound. If those had to sync, the app would need accounts — the subject of the scaling chapter.

There is one subtle correctness rule the code follows: because translated text has different characters, highlights are keyed by language (`highlights-<id>-<lang>`), and the translate route refuses to return a paragraph count that does not match. Together these keep a stored offset always pointing at the same characters it was created against.

11.4 · Caching as a first-class decision

Every expensive result is stored and reused. The pattern appears at three scales:

- **Transcripts** — cached forever in Postgres. The transcribe route's very first act after validation is to check the cache; a hit skips Supadata and Gemini entirely. This makes transcription a once-per-video cost shared across all users.
- **Translations** — cached per language in memory and in `localStorage`, so switching languages back and forth is instant after the first time.
- **Summaries** — reuse an already-saved transcript's text instead of re-fetching captions.

The price of caching is staleness: a transcript keeps its original title even if the video is later renamed, because nothing invalidates the cache. The project accepts this because titles rarely change and the savings are large. Knowing *where* staleness can appear is part of understanding the system.

11.5 · Failure policy: degrade or refuse, never lie

Two distinct failure strategies are used, chosen per situation:

- **Degrade** when a rougher answer is still useful. If Gemini cleanup fails, the transcribe route falls back to heuristically grouped paragraphs. If the free dictionary has no entry, the reader falls back to Gemini. The user still gets something.

- **Refuse** when a wrong answer would be worse than none. If the translation comes back with a different number of paragraphs, the route returns an error rather than risk misaligned highlights. If `localStorage` is corrupt, the reader treats it as empty rather than crash.

The deciding question is always: "would a partial or approximate result mislead the user or corrupt stored data?" If yes, refuse; if no, degrade. This single principle explains nearly every `try/catch` and every early `return null` in the codebase.

11.6 · Measure, don't hardcode

Several effects that look purely visual are actually small measurement engines. The title underline measures each wrapped line with `Range.getClientRects()` and draws a bar per line, because CSS underlines cannot carry a separate shadow color. The library's hover underline measures the last line so it hugs the text. The home page measures the headline to size the input row. The loading page computes its minimum visible time from the animation constants rather than picking a number. The common thread: the code reads the real, laid-out page and reacts to it, so the result stays correct when fonts load, text wraps differently, or the window resizes. The cost is complexity and a dependence on running in a real browser (these effects do nothing on the server).

12 · Architectural decisions, argued

The first edition tabulated the decisions. Here each is argued: the alternative that was rejected, and the condition under which the decision would flip.

One Next.js application instead of a separate API server

Alternative rejected: a standalone backend (say, an Express or FastAPI service) with the React app calling it across a network boundary. **Why the monolith won:** a single codebase, a single deploy, shared TypeScript types across front and back, and secrets that live naturally in server-only files. **When it would flip:** if the backend needed to run long jobs, scale independently of the UI, or be consumed by non-web clients, a separate service (or at least a job worker) becomes worth the operational cost. The transcription time limit is the first pressure in that direction.

Postgres for transcripts, `localStorage` for the personal layer

Alternative rejected: full user accounts from day one, with everything in the database. **Why the split won:** it removed the entire authentication and account-management surface while still delivering shareable transcripts and a working personal experience on a single device. **When it would flip:** the moment two people must use the same deployment, or one person must see their highlights on two devices. That is a single, well-scoped migration (Chapter 15), and the code is arranged to make it tractable — all local access is funneled through `highlights.ts`.

Transcribe once, cache permanently

Alternative rejected: re-transcribing on each view, or a time-boxed cache. **Why permanent won:** transcription is the only slow, paid step; the content of a video does not change; the savings compound with every repeat view across all users. **When it would flip:** if transcripts needed to reflect edited captions or updated metadata, the cache would need an invalidation signal (a "re-clean" action or a background refresh).

The AI as an optional stage, driven by a prompt

Alternative rejected: hard dependence on the model, or hand-written cleanup rules in code. **Why the current design won:** the fallback keeps the app alive when the model is down, and expressing the rules as a prompt lets the author add corrections (misheard words, the all-caps fix) without touching code or redeploying logic — only the string changes. **When it would flip:** if the correction list grew unwieldy or needed to be exact and testable, a deterministic find/replace pass would move those rules back into code, leaving the model for the genuinely fuzzy work.

Four runtime dependencies; write the rest

Alternative rejected: pulling in libraries for OAuth, HTTP clients, UI components, state management. **Why hand-writing won:** a small, auditable dependency tree; fast builds; no version churn; full control over behavior. The OAuth flow, the dropdown, the wave math, and the reader's selection handling are all written directly. **When it would flip:** when a hand-written piece becomes a maintenance burden or a security-sensitive area (real multi-user OAuth is a strong candidate for a vetted library).

Tailwind with heavy use of arbitrary values

Alternative rejected: a conventional CSS file with named component classes. **Why Tailwind-inline won:** the design was tuned pixel by pixel through many iterations, and inline classes keep each adjustment next to the

element it affects, with no naming overhead. **The cost, stated plainly:** class strings are long, and the design lives in the markup rather than a stylesheet — a reader must look at the JSX to see the styling. For this project's iteration-heavy history, that trade paid off.

13 · The limits, examined

Each limit below is stated, then explained mechanically — *why* it exists in the code — so you can judge how hard it would be to lift.

Single user

Mechanism: the Google refresh token is stored in one `httpOnly` cookie on one browser; there is no user table and no session identity. Any visitor to the deployment shares that one connection. **To lift:** introduce accounts and a per-user token store (Chapter 15). This is foundational, not cosmetic.

Personal data is device-bound

Mechanism: highlights, notes, preferences, progress, and dismissals are written to `localStorage`, which is per-browser by definition. **Consequence to internalize:** the library's counts and percentages describe *this* browser's activity, not a global truth. **To lift:** move these into per-user database tables — the same migration as accounts.

Captions are mandatory

Mechanism: Supadata returns nothing for a video without captions, and the route answers 422. Summary and translation build on the transcript, so they inherit the requirement. **To lift:** add speech-to-text (transcribe the audio directly), a substantially larger and costlier pipeline.

English-first

Mechanism: captions are requested with `lang=en`; translation targets are limited to French and Spanish in the route's `LANGS` map; the free dictionary is reliable mainly for English. **To lift:** detect the source language, request the matching captions, and expand the target and dictionary handling.

The five-minute ceiling

Mechanism: transcription runs inside one serverless function whose limit is raised to 300 seconds; a very long video's caption fetch plus AI pass can approach it, and there is no way to continue past it. **To lift:** move transcription to a background job (Chapter 15).

Word-level timing is discarded

Mechanism: cleanup regroups text into paragraphs and sets each segment's `startMs` to `0`, because the original per-fragment timings no longer align with the rewritten paragraphs. **Consequence:** the reader cannot jump from a paragraph to that moment in the video. **To lift:** carry and re-anchor timings through cleanup, or keep a parallel timed copy.

Feed is quota-bound and capped

Mechanism: the YouTube Data API has a daily quota; `getFeed` caps channels (~100), the window (30 days), and drops Shorts to stay within it, and fetches durations in batches. A very large subscription set is truncated. **To lift:** cache feed results, request a higher quota, or precompute feeds on a schedule rather than on demand.

No automated tests; metadata can go stale; cost scales with new videos

These three share a theme: the project optimizes for iteration speed over operational safety. Correctness is checked by building and by manual verification; a cached title can drift from a renamed video; each first-time transcription spends credits with no per-user budget. None blocks current use; all matter at scale, and all are addressed in the next chapter.

14 · Scaling, as an engineering plan

The first edition sketched the phases. Here each phase is turned into concrete work: what to build, which files change, and what could go wrong.

14.1 · Phase A — accounts and a synced personal layer

Goal: multiple users; personal data that follows each user across devices. **Build:** a real sign-in (extend the existing Google OAuth to issue a session for every user, not just a feed token); a `users` table; and per-user tables for highlights, notes, preferences, and progress. **Files that change:** the four `auth/google` routes gain a session concept; `highlights.ts` is reimplemented (or shadowed) to read and write the server instead of `localStorage`, ideally behind the same function names so callers are unaffected; the reader and library switch from local reads to fetches. **Risks:** Google requires OAuth verification before the public can sign in; migrating existing `localStorage` data into accounts needs a one-time import; you now store personal data, which raises privacy and deletion obligations.

14.2 · Phase B — transcription as a background job

Goal: remove the time ceiling; enable auto-transcription. **Build:** a `jobs` table (video id, status pending/processing/done/failed, result); an enqueue step that replaces the synchronous call; a worker (a scheduled function or a small dedicated service) that pulls pending jobs, runs the existing fetch-and-clean pipeline, and writes results; and a client that shows "processing" and polls or subscribes. **Files that change:** the transcribe route splits into "enqueue" and "process"; the loading page becomes a status page; the feed's "move to library" enqueues instead of blocking. **Risks:** a worker introduces the first piece of always-on infrastructure; you must handle retries, stuck jobs, and duplicate enqueues; ordering and idempotency now matter.

14.3 · Phase C — cost, abuse, and correctness controls

Goal: keep spending and quality bounded as usage grows. **Build:** per-user rate limits and a spend cap; a negative cache (remember videos known to lack captions, so they are not retried); a deterministic find/replace pass for the known-mistranscription list, leaving the model for fuzzy work; and an automated test suite around the parsing helpers, the URL/id extraction, the paragraph grouping, and the translation-alignment check. **Files that change:** a thin middleware in front of the paid routes; a small corrections module imported by `cleanup.ts`; a `tests/` tree. **Risks:** rate limits need shared state (a store), which is new infrastructure; tests must run in CI to be worth anything.

14.4 · Phase D — hardening and reach

Goal: operate reliably and serve more content. **Build:** a real migration tool (replace the lazy `ALTER TABLE` in `db.ts`); database indexes as the table grows; monitoring and alerting on the external services; then the additive reach features — non-English source captions, more translation targets, and word-level timings that let the reader link a paragraph to a moment in the embedded player. **Why this order:** reach features are the easy layer; they are placed last precisely because they do not require the foundational work, so they can wait until the system beneath them is solid.

The load-bearing observation. The current code already isolates the things Phases A–D must change: all storage is in `db.ts` and `highlights.ts`; all external calls are in `lib/` and the routes; the UI reads through small functions. That isolation is why the plan above is a series of contained migrations rather than a rewrite.

15 · Glossary (expanded)

Every term from the first edition, plus the ones introduced in Part I's later chapters and the concepts this edition adds.

Term	Meaning
anchor (URL `#name`)	A fragment on a URL that points at an element with that `id`; used to deep-link into the highlights/notes sections.
API endpoint / route	A server URL that returns data, not a page. The `route.ts` files.
async / await / Promise	Machinery for waiting on slow work without freezing the single thread. A Promise is a value-to-come; `await` pauses one function until it settles.
callback	A function passed to be called later — by an event, a timer, or when a promise settles.
client component	A React component that runs in the browser; marked `'use client'`.
cookie / `httpOnly`	A value the browser stores and returns with each request. `httpOnly` means page scripts cannot read it — used for the Google token so a script cannot steal it.
CRUD	Create, Read, Update, Delete.
debounce / defer (`setTimeout(…, 0)`)	Postponing work to a later tick so other handlers (e.g. a click on a highlight) run first.
dependency array	The second argument to `useEffect` / `useMemo`: the values that, when changed, re-run the hook. `[]` means "once."
destructuring	Pulling fields out of an object/array by name or position.
discriminated union	A union of object types distinguished by a shared tag field (here, `popup.kind`).
DOM	The live, in-memory tree of the page's elements that scripts read and change.
environment variable	A named secret set in the deployment, read with `process.env`, never in the code.
event loop / single thread	The model where one thread runs JS and slow work is handed off and resumed via queued callbacks.
fetch	The built-in function for making an HTTP request.
hook	A React function starting with `use` that gives a component memory or behavior.
hydration	The browser attaching interactivity to server-rendered HTML when a client component loads.
idempotent	An operation that is safe to repeat with the same effect (relevant to job queues).
JSX	HTML-like syntax inside JavaScript describing what a component renders.
localStorage / sessionStorage	Browser string stores. `local` persists; `session` clears with the tab.
migration	A change to the database schema applied to an existing database (here, the lazy `ALTER TABLE`).
OAuth / refresh token / access token	The flow letting an app use your account elsewhere without your password. A long-lived refresh token is exchanged for short-lived access tokens.

optional chaining `?.` / nullish `??`	Read only if the parent exists; use the left value unless null/undefined.
ref	A mutable box (`useRef`) that survives re-renders without causing one; often a handle to a DOM element.
regular expression	A text-matching pattern written between slashes.
render / re-render	A component producing its JSX; React re-runs it when its state or props change.
serverless function	Server code that runs on demand, briefly, with a time limit — how the routes are deployed.
SQL / upsert	The database query language; upsert = insert-or-update (`ON CONFLICT DO UPDATE`).
SVG / path `d`	Vector graphics; a shape described by a string of move/line/curve commands.
tagged template literal	A backtick string processed by a function, e.g. `sql`…`` , which safely inserts values.
ternary `?:`	An inline if/else that produces a value.
TreeWalker	A browser object that visits DOM nodes in order; used to compute character offsets.
type / interface / union	TypeScript shapes checked before runtime; a union is "one of several types."

16 · Syntax appendix (expanded)

The first edition's cheat-sheet, plus the constructs Part II's annotations rely on. Skim now; return when a line of code is opaque.

The pieces from the first edition (recap)

```
const x = 5; let y = 10                    // immutable vs reassignable binding
const f = (a: number): number => a + 1     // typed arrow function
const { name } = obj; const [a, b] = arr   // object / array destructuring
const copy = { ...obj, age: 4 }            // spread with override
arr.map / .filter / .sort / .join / str.split  // array & string transforms
cond ? a : b   value ?? d   obj?.f?.g   !!v // ternary, nullish, optional chain, coerce
```

Truthiness

```
// "Falsy" values: false, 0, '', null, undefined, NaN. Everything else is "truthy".
if (!text.trim()) return          // empty-after-trim string is falsy
arr.filter(Boolean)               // drops falsy items (e.g. empty strings)
const n = raw ? parseInt(raw) : 0 // guard before parsing
```

Closures and the event-listener cleanup pattern

```
useEffect(() => {
  const onKey = (e: KeyboardEvent) => { if (e.key === 'Escape') close() }
  document.addEventListener('keydown', onKey)   // subscribe
  return () => document.removeEventListener('keydown', onKey)  // cleanup on unmount/re-
run
}, [close])
```

The inner `onKey` "closes over" `close` — it remembers that variable. The returned function is the effect's cleanup; React runs it before re-running the effect and when the component unmounts. Removing exactly the function you added is why `onKey` is named, not inline.

Immutable updates (why the code never mutates state directly)

```
// WRONG: mutates the existing array; React may not notice.
highlights.push(h)
// RIGHT: create a new array; React sees a new reference and re-renders.
persist([...rest, merged])
setNote: persist(highlights.map(h => h.id === id ? { ...h, note } : h))
```

React decides whether to re-render by comparing references. A new array/object signals "changed." This is why you see `[...list, x]`, `.map(...)`, and `{ ...obj, field }` instead of `push`, index assignment, or `obj.field =`.

Discriminated unions and narrowing

```
type Popup =
  | { kind: 'actions'; spans: Span[]; text: string; isWord: boolean }
  | { kind: 'note'; highlightId: string }
  | { kind: 'define'; word: string; loading: boolean; result: DefineResult | null }
  | null

if (popup?.kind === 'define') {
  // Inside here TypeScript KNOWS popup has `word`, `loading`, `result`.
}
```

Checking the tag (`kind`) "narrows" the type: after the check, the compiler lets you use only the fields that variant has. This is how one `popup` variable safely represents three different pop-ups.

Rendering lists and the `key` rule

```
{segments.map((seg, i) => (
  <p key={i} data-p={i}>{renderParagraph(seg.text, ...)}</p>
))}
```

Every element in a mapped list needs a stable `key` so React can tell which item is which across re-renders. `data-p={i}` is a **data attribute** — custom metadata on an element, later read with `element.dataset.p` to recover the paragraph index during selection handling.

Conditional rendering

```
{loading && <p>Loading…</p>}                 // render only if loading is truthy
{items.length === 0 ? <Empty/> : <List/>}     // one or the other
{value && (() => { /* compute */; return <X/> })()}  // inline IIFE for logic before
returning JSX
```

JSX has no `if` statement inside it, so conditions are expressed with `&&` , ternaries, or — when you need statements — an immediately-invoked function (the `(() => {...})()` form the note pop-up uses).

17 · How to use Part II

Part I explained the project. Part II reads two files the way you would in a code review: top to bottom, in order, with the reasoning for each block. These two files — the transcript reader and the transcript page — together contain most of the project's genuinely hard client-side logic. If you understand them, the rest of the codebase is straightforward by comparison.

Open the real file in your editor alongside this chapter. The line numbers here match the source at the time of writing; if the file has drifted, match on the code, not the number. Each block is quoted, then explained. Where a construct was covered in Chapter 16, it is named rather than re-explained — flip back if needed.

> **Read actively.** After each block, look away and try to state in one sentence what it does and why. The trace-it-yourself exercises in Chapter 20 assume you have done this.

18 · Line-by-line: `src/components/TranscriptReader.tsx`

The reading surface. It renders paragraphs and layers highlighting, notes, and dictionary lookup over them. About 500 lines, read here in eleven blocks.

Block 1 — the directive and imports (lines 1–9)

```
'use client'

import { Fragment, useCallback, useEffect, useMemo, useRef, useState } from 'react'
import { Highlight, loadHighlights, saveHighlights, charOffsetWithin } from
'@/lib/highlights'
```

`'use client'` makes this a browser component — mandatory, because it uses state, effects, refs, and the DOM. The React import pulls in the hooks it needs plus `Fragment` (a wrapper that groups elements without adding a real DOM node — used when a function must return several pieces of text side by side). The second import is the project's own highlight utilities from Chapter 6.3: the `Highlight` type, load/save, and the offset helper. Importing exactly what is used keeps dependencies visible.

Block 2 — the data shapes (lines 11–47)

```
interface Segment { text: string; startMs: number }

interface Props {
  transcriptId: string
  lang: string            // 'en' | 'fr' | 'es' -- drives which dictionary to query
  segments: Segment[]
  fontSizeRem: number; lineHeight: number; fontFamily: string
  isDark: boolean
}

interface Span { paragraph: number; start: number; end: number; text: string }

type Popup =
  | { kind: 'actions'; x: number; y: number; spans: Span[]; text: string; isWord: boolean
}
  | { kind: 'note'; x: number; y: number; highlightId: string }
  | { kind: 'define'; x: number; y: number; word: string; loading: boolean; result:
DefineResult | null }
  | null

interface DefineResult { word: string; phonetic?: string; meanings: { partOfSpeech:
string; definition: string }[] }
```

`Props` is the contract with the parent (the transcript page): it passes the transcript id, language, the paragraphs, and the resolved reading styles (size, line height, font) plus whether the theme is dark. The component receives style values rather than computing them, so all reading-settings logic stays in the page.

`Span` is a per-paragraph slice of a selection — the unit that becomes a highlight. `Popup` is the discriminated union from Chapter 16: one variable, four possibilities (three pop-ups plus `null`). Storing all pop-up state in one place guarantees only one is open and lets the render code switch on `kind`. `DefineResult` shapes the dictionary answer; `phonetic?` is optional because not every source provides it.

Block 3 — constants and the look-up test (lines 26–51)

```
const SF_FONT = '-apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif'
const SERIF_FONT = 'var(--font-serif-family), serif'
const MINT = '#54FFC9'

const isLookupable = (s: string) =>
  /^[\p{L}\p{M}''-]+(?:\s[\p{L}\p{M}''-]+)?$/u.test(s.trim())
```

Two font stacks and the highlight color are named once. `isLookupable` decides whether a selection can go to the dictionary. The regular expression, read piece by piece: `^` start; `[\p{L}\p{M}''-]+` one or more letters (`\p{L}` is any Unicode letter, `\p{M}` a combining accent mark), apostrophes, or hyphens; `(?:\s…)?` an optional second such word after a space; `$` end; the `u` flag enables Unicode property escapes. In words: one word, or two words — so "teed up" and "New York" qualify, but a whole sentence does not. This is why the Dictionary button appears only for short selections.

Block 4 — state and persistence (lines 53–77)

```
const [highlights, setHighlights] = useState<Highlight[]>([])
const [popup, setPopup] = useState<Popup>(null)
const containerRef = useRef<HTMLDivElement>(null)

useEffect(() => { setHighlights(loadHighlights(transcriptId, lang)) }, [transcriptId,
lang])

const persist = useCallback((next: Highlight[]) => {
  setHighlights(next)
  saveHighlights(transcriptId, lang, next)
}, [transcriptId, lang])
```

Two pieces of state — the highlight list and the current pop-up — plus a ref to the paragraphs container (needed later to scope DOM queries and measure the selection). The effect loads highlights whenever the transcript or language changes; because the storage key includes the language, switching to French loads a different set. `persist` is the one function that changes highlights: it updates state *and* writes to `localStorage` together, so the two never diverge. It is wrapped in `useCallback` so its identity is stable across renders (it changes only if the id or language does) — relevant because effects depend on it.

Block 5 — selection to spans (lines 80–126)

```
const handleMouseUp = useCallback(() => {
  setTimeout(() => {
    const sel = window.getSelection()
    if (!sel || sel.isCollapsed || sel.rangeCount === 0) return
    const range = sel.getRangeAt(0)
    const text = sel.toString()
    if (!text.trim()) return

    const root = containerRef.current
    if (!root) return
    const elFor = (node: Node): HTMLElement | null =>
      (node.nodeType === Node.TEXT_NODE
        ? node.parentElement?.closest('[data-p]')
        : (node as HTMLElement).closest?.('[data-p]')) as HTMLElement | null
    const startPara = elFor(range.startContainer)
    const endPara   = elFor(range.endContainer)
    if (!startPara || !endPara) return

    const paraEls = Array.from(root.querySelectorAll<HTMLElement>('[data-p]'))
    const spans: Span[] = []
    for (const el of paraEls) {
      if (!range.intersectsNode(el)) continue
      const paragraph = Number(el.dataset.p)
      const full = segments[paragraph]?.text.length ?? 0
      const start = el === startPara ? charOffsetWithin(el, range.startContainer,
range.startOffset) : 0
      const end   = el === endPara   ? charOffsetWithin(el, range.endContainer,
range.endOffset)   : full
      const a = Math.min(start, end), b = Math.max(start, end)
      if (a >= b) continue
      spans.push({ paragraph, start: a, end: b, text: segments[paragraph].text.slice(a, b)
})
    }
    if (spans.length === 0) return

    const rect = range.getBoundingClientRect()
    setPopup({ kind: 'actions', x: rect.left + rect.width / 2, y: rect.top, spans,
               text: text.trim(), isWord: spans.length === 1 && isLookupable(text) })
  }, 0)
}, [])
```

This is the hardest function in the file; take it slowly. It runs when the mouse is released over the text. The `setTimeout(…, 0)` defers everything by one tick so that if the release was actually a click on an existing highlight, that click's own handler runs first (see Block 9). Inside: read the browser's **selection**; bail if it is empty or collapsed. Get the **range** (the selection's start and end position in the DOM) and its text.

`elFor` finds, for a given DOM node, the paragraph element it belongs to. A selection endpoint may land on a text node or an element; either way, `.closest('[data-p]')` climbs to the nearest ancestor carrying the `data-p` attribute — the paragraph marker set in the render. If either endpoint is not inside a paragraph, bail.

Then the loop: over every paragraph element, skip those the range does not touch (`range.intersectsNode`). For each one it does touch, compute a start and end *character offset*. For the paragraph where the selection starts, the start offset is measured with `charOffsetWithin` (Chapter 6.3, which flattens the possibly-split text nodes into one offset); for paragraphs in the middle the selection covers the whole thing, so start is `0` and end is the full length; the paragraph where it ends contributes a real end offset. `Math.min/max` orders them (a selection

can be made right-to-left), and a zero-width span is skipped. Each surviving slice becomes a `Span` with a copy of its text.

Finally, if any spans exist, open the `actions` pop-up centered above the selection (from `range.getBoundingClientRect()`), marking it `isWord` only when the whole selection is a single span and passes `isLookupable` — that flag is what shows or hides the Dictionary button. The empty dependency array `[]` means this callback is created once; it reads `segments` from the surrounding scope, which is stable for a given render.

Block 6 — dismissing the pop-up (lines 129–146)

```
useEffect(() => {
  if (!popup) return
  const onDown = (e: MouseEvent) => {
    const t = e.target as HTMLElement
    if (t.closest('[data-reader-popup]') || t.closest('[data-hl]')) return
    setPopup(null)
  }
  const onKey = (e: KeyboardEvent) => e.key === 'Escape' && setPopup(null)
  const onScroll = () => setPopup((p) => (p && p.kind === 'actions' ? null : p))
  document.addEventListener('mousedown', onDown)
  document.addEventListener('keydown', onKey)
  window.addEventListener('scroll', onScroll, { passive: true })
  return () => { /* remove all three */ }
}, [popup])
```

Only active while a pop-up is open (`if (!popup) return`). It closes the pop-up on an outside click — unless the click was inside the pop-up itself (`[data-reader-popup]`) or on a highlight (`[data-hl]`), which have their own behavior — on Escape, and, for the transient actions pop-up only, on scroll (the note and dictionary pop-ups survive scrolling because `onScroll` returns them unchanged). The cleanup removes all three listeners; because the effect depends on `popup`, the listeners are re-bound whenever the pop-up changes, always closing over the current value. This is the cleanup pattern from Chapter 16.

Block 7 — creating and merging highlights (lines 151–186)

```
const mergeSpan = (list, span) => {
  const { paragraph, start, end } = span
  const overlapping = list.filter(h => h.paragraph === paragraph && h.start < end && h.end
> start)
  const mergedStart = Math.min(start, ...overlapping.map(h => h.start))
  const mergedEnd   = Math.max(end,   ...overlapping.map(h => h.end))
  const keptNote = overlapping.map(h => h.note).filter(Boolean).join('\n')
  const rest = list.filter(h => !overlapping.includes(h))
  const merged = { id:
`${Date.now()}-${Math.random().toString(36).slice(2,7)}-${paragraph}`,
                   paragraph, start: mergedStart, end: mergedEnd,
                   text: segments[paragraph].text.slice(mergedStart, mergedEnd),
                   note: keptNote, createdAt: Date.now() }
  return { list: [...rest, merged], merged }
}

const addHighlights = (spans) => {
  let list = highlights
  for (const span of spans) list = mergeSpan(list, span).list
  persist(list)
  window.getSelection()?.removeAllRanges()
}
```

`mergeSpan` folds one new span into the list. Two ranges in the same paragraph overlap when each starts before the other ends (`h.start < end && h.end > start` — the standard interval-overlap test). All overlapping highlights are gathered, their extent unioned with the new span (`Math.min/max` spread over their starts/ends), any notes concatenated, and the originals removed (`rest`); one `merged` highlight replaces them. The id combines a timestamp, a short random string, and the paragraph index to be unique. `addHighlights` applies this for each span of a multi-paragraph selection, persists once at the end, and clears the browser selection so the highlight is visible without the blue selection overlay. Note the immutable style throughout (Chapter 16): new arrays, never mutation.

Block 8 — remove, note, and the dictionary (lines 179–237)

```
const removeHighlight = (id) => { persist(highlights.filter(h => h.id !== id));
setPopup(null) }
const setNote = (id, note) => persist(highlights.map(h => h.id === id ? { ...h, note } :
h))

const define = async (word, x, y) => {
  setPopup({ kind: 'define', x, y, word, loading: true, result: null })
  let result = null
  try { /* GET dictionaryapi.dev; take up to 3 meanings if found */ } catch {}
  if (!result) {
    try { /* POST /api/define (Gemini fallback) */ } catch {}
  }
  setPopup(p => (p && p.kind === 'define' && p.word === word ? { ...p, loading: false,
result } : p))
}
```

`removeHighlight` and `setNote` are one-liners built on `persist` — remove by filtering out an id; edit a note by mapping and replacing one highlight with a copy carrying the new note. `define` is asynchronous: it opens the dictionary pop-up in a loading state immediately (so the user sees a response at once), tries the free dictionary API directly from the browser, and only if that yields nothing falls back to the project's `/api/define` Gemini

route (Chapter 7.4). The final `setPopup` uses the functional form and re-checks `kind === 'define' && word === word` so a slow answer for an old word cannot overwrite a newer pop-up — a guard against a race between two quick look-ups.

Block 9 — rendering a paragraph with highlights (lines 240–306)

```
const highlightsByPara = useMemo(() => {
  const map = new Map<number, Highlight[]>()
  for (const h of highlights) { const arr = map.get(h.paragraph) ?? []; arr.push(h);
map.set(h.paragraph, arr) }
  return map
}, [highlights])

const renderParagraph = (text, paraHighlights) => {
  const nameMatch = text.match(/^([^:]{1,40}):\s+/)
  const nameEnd = nameMatch ? nameMatch[1].length + 1 : 0
  const renderPlain = (a, b, keyPrefix) => { /* bold the "Speaker:" prefix if this slice
covers it */ }
  if (paraHighlights.length === 0) return renderPlain(0, text.length, 'p')

  const points = new Set<number>([0, text.length])
  paraHighlights.forEach(h => { points.add(Math.max(0, h.start));
points.add(Math.min(text.length, h.end)) })
  const sorted = [...points].sort((a, b) => a - b)
  const out = []
  for (let i = 0; i < sorted.length - 1; i++) {
    const a = sorted[i], b = sorted[i + 1]; if (a >= b) continue
    const h = paraHighlights.find(hh => hh.start <= a && hh.end >= b)
    if (h) out.push(<mark data-hl={h.id} onClick={openNotePopup} …>{renderPlain(a, b, …)}
</mark>)
    else   out.push(<Fragment>{renderPlain(a, b, …)}</Fragment>)
  }
  return out
}
```

First, `highlightsByPara` groups the flat highlight list into a map from paragraph index to that paragraph's highlights, recomputed only when highlights change (`useMemo`). Then `renderParagraph` turns one paragraph's text plus its highlights into React nodes. It cannot simply print the text, because the highlighted ranges must become `<mark>` elements.

The algorithm is "slice at every boundary." It builds a set of break points — the paragraph's start and end, plus every highlight's start and end (clamped into range) — sorts them, and walks adjacent pairs. For each slice it asks whether a highlight fully covers it; if so it emits a `<mark>` (mint background, an inset purple underline when the highlight carries a note, a click handler that opens the note pop-up, and a `data-hl` id so Block 6 knows a click landed on a highlight), otherwise a plain fragment. `renderPlain` additionally bolds a leading "Speaker:" label: it detects a short prefix ending in a colon and renders that portion bold-italic wherever a slice overlaps it. The result is an array of nodes that reconstructs the paragraph with highlights wrapped and speaker labels styled.

Block 10 — the two chronological sections (lines 308–408)

```
const highlightsChrono = useMemo(() => [...highlights].sort((a, b) => a.createdAt -
b.createdAt), [highlights])
const notesChrono = useMemo(() => highlightsChrono.filter(h => h.note.trim()),
[highlightsChrono])

useEffect(() => {
  const hash = window.location.hash.replace('#', '')
  if (hash !== 'highlights' && hash !== 'notes') return
  requestAnimationFrame(() => document.getElementById(hash)?.scrollIntoView({ behavior:
'smooth', block: 'start' }))
}, [highlightsChrono.length, notesChrono.length])
```

Two derived lists: all highlights in creation order, and just those with a note. Each is memoized. The effect implements the deep link from the library: if the page URL ends in `#highlights` or `#notes`, it scrolls to that section once it has rendered (`requestAnimationFrame` waits for the next paint so the target exists). It depends on the section lengths so it fires after the lists appear. The JSX for both sections (omitted here) lists each entry as a quoted blockquote plus an editable note textarea and a Remove button, and gives the sections the `id="highlights"` / `id="notes"` anchors those links target.

Block 11 — the floating pop-ups (lines 411–505)

```
{popup?.kind === 'actions' && ( /* Highlight | Dictionary buttons, positioned at popup.x/y
*/ )}
{popup?.kind === 'note' && (() => { const h = highlights.find(x => x.id ===
popup.highlightId); … })()}
{popup?.kind === 'define' && ( /* word, phonetic, meanings, or "Looking it up…" / "No
definition found." */ )}
```

The render ends by drawing whichever pop-up is open, selected by `popup.kind` (Chapter 16's narrowing: inside each branch the correct fields are available). All three are `position: fixed` at the coordinates stored when the pop-up opened, translated so they sit centered above the selection or highlight, and marked `data-reader-popup` so Block 6's outside-click check skips them. The actions pop-up shows Highlight always and Dictionary only when `isWord`. The note pop-up is written as an immediately-invoked function because it needs a statement (look up the highlight, bail if gone) before returning JSX. The define pop-up switches on `loading` and whether a result was found. This block is where all the earlier state finally becomes visible interface.

What Block-by-block reading revealed. The file is organized as: shapes, then state, then event-to-data (selection→spans), then data operations (merge/remove/note/define), then data-to-view (render paragraphs, render sections, render pop-ups). That "shapes → state → inputs → operations → output" order is a template you can impose on any component you write.

19 · Line-by-line: `src/app/transcript/[id]/page.tsx`

The reader page — the coordinator. It fetches or reads the transcript, applies reading settings, handles translation, remembers scroll position, draws the per-line title underline, exposes copy/download/delete, and hands the text to the `TranscriptReader` of Chapter 18. About 500 lines, read in nine blocks.

Block 1 — imports and the synchronous local read (lines 1–37)

```
'use client'
import { useEffect, useLayoutEffect, useRef, useState } from 'react'
import { useParams, useRouter } from 'next/navigation'
import Link from 'next/link'
import { clearLocalTranscript, loadProgress, saveProgress } from '@/lib/highlights'
import ReadingSettingsMenu, { useReadingPrefs, THEMES, FONTS, SIZE_STEPS, SPACING_STEPS,
WIDTH_STEPS } from '@/components/ReadingSettingsMenu'
import TranscriptReader from '@/components/TranscriptReader'

function readLocalTranscript(id: string): { segments: Segment[]; title: string } | null {
  if (typeof window === 'undefined') return null
  try {
    const raw = localStorage.getItem(`transcript-${id}`)
    if (!raw) return null
    const parsed = JSON.parse(raw)
    const segs = Array.isArray(parsed) ? parsed : parsed.segments
    if (!segs) return null
    return { segments: segs, title: Array.isArray(parsed) ? '' : parsed.title ?? '' }
  } catch { return null }
}
```

A client page. `useParams` reads the dynamic `[id]` from the URL; `useRouter` lets the code navigate programmatically; `Link` is the client-side navigation element for the header. It imports the reading-settings hook and its catalogs, and the reader component. `readLocalTranscript` reads this device's cached copy synchronously, tolerating both the old storage shape (a bare array) and the new one (`{ segments, title }`) — a small piece of backward compatibility. It guards with `typeof window` so it is safe if evaluated on the server, and swallows parse errors.

Block 2 — state, with lazy initializers (lines 39–67)

```
const params = useParams(); const router = useRouter(); const id = params.id as string

const handleDelete = async () => {
  if (!window.confirm('Delete this transcript from your library? This can’t be undone.'))
return
  clearLocalTranscript(id)
  try { await fetch(`/api/transcripts/${id}`, { method: 'DELETE' }) } catch {}
  router.push('/library')
}

const [segments, setSegments] = useState<Segment[]>(() =>
readLocalTranscript(id)?.segments ?? [])
const [title, setTitle]       = useState(() => readLocalTranscript(id)?.title ?? '')
const [loading, setLoading]   = useState(() => readLocalTranscript(id) === null)
const [prefs, setPrefs] = useReadingPrefs()
/* plus: notFound, copied, settingsOpen, downloadOpen, headerHidden, translations,
translating, translateError, titleRef, titleLineRects */
```

The important detail is the **lazy initializer**: `useState(() => …)` passes a function, so the local read runs once, during the first render, not on every render. Because `segments` and `title` start already populated from `localStorage`, the common arrival from the loading page shows the transcript immediately, and `loading` starts `false` whenever a local copy exists — no spinner flash. `handleDelete` confirms, clears the local cache (Chapter 6.3), deletes the database row, and navigates to the library. `useReadingPrefs` (Chapter 8.2) supplies the persisted reading settings.

Block 3 — the title's per-line underline (lines 79–98)

```
useLayoutEffect(() => {
  const measure = () => {
    const el = titleRef.current; if (!el || !el.firstChild) return
    const range = document.createRange(); range.selectNodeContents(el)
    const parentRect = el.getBoundingClientRect()
    const rects = Array.from(range.getClientRects())
    setTitleLineRects(rects.map(r => ({ left: r.left - parentRect.left, width: r.width,
bottom: r.bottom - parentRect.top })))
  }
  measure(); window.addEventListener('resize', measure)
  return () => window.removeEventListener('resize', measure)
}, [displayTitle, prefs])
```

The measure-don't-hardcode pattern (Chapter 11.6) in full. A CSS underline cannot have its own shadow color, so the page draws bars itself. `useLayoutEffect` (not `useEffect`) is used because it runs before the browser paints, avoiding a flicker of un-underlined title. It makes a DOM `Range` over the title's contents and calls `getClientRects()`, which returns one rectangle per *wrapped line*. Each rectangle is converted to coordinates relative to the title element and stored; the render draws a purple bar at each. It re-measures on window resize and whenever the title text or preferences change (a bigger font wraps differently).

Block 4 — header hide and scroll-progress save (lines 100–119)

```
useEffect(() => {
  let lastY = window.scrollY
  const onScroll = () => {
    const y = window.scrollY
    if (y > 80 && y > lastY) setHeaderHidden(true)
    else if (y < lastY) setHeaderHidden(false)
    lastY = y
    const scrollable = document.documentElement.scrollHeight - window.innerHeight
    saveProgress(id, scrollable > 0 ? (y / scrollable) * 100 : 0)
  }
  window.addEventListener('scroll', onScroll, { passive: true })
  return () => window.removeEventListener('scroll', onScroll)
}, [id])
```

One scroll listener does two jobs. It hides the header when scrolling down past 80px and reveals it when scrolling up (comparing the new position to `lastY`). And it computes reading progress as a percentage — current scroll over the total scrollable height — and writes it with `saveProgress` (Chapter 6.3). `{ passive: true }` tells the browser the listener will not block scrolling, keeping it smooth. This is the write half of the progress feature; Block 5 is the read half.

Block 5 — restoring the scroll position once (lines 121–142)

```
const restoredRef = useRef(false)
useEffect(() => {
  if (loading || notFound || restoredRef.current) return
  if (window.location.hash === '#highlights' || window.location.hash === '#notes') {
restoredRef.current = true; return }
  const pct = loadProgress(id)
  if (pct <= 0) { restoredRef.current = true; return }
  restoredRef.current = true
  const restore = () => {
    const scrollable = document.documentElement.scrollHeight - window.innerHeight
    if (scrollable > 0) window.scrollTo({ top: (pct / 100) * scrollable })
  }
  requestAnimationFrame(restore)
  document.fonts?.ready?.then(() => requestAnimationFrame(restore))
}, [loading, notFound, id])
```

A guard ref (`restoredRef`) ensures restoration happens at most once. It defers to a deep link (if the URL asks for a section, that scroll wins) and does nothing if there is no saved progress. Otherwise it scrolls to the saved fraction of the page height — twice: once on the next animation frame, and again after web fonts finish loading (`document.fonts.ready`), because fonts can change the page height and move the target. This double-restore is a small robustness measure against layout shifting under the restore.

Block 6 — load from the database; derive the display copy (lines 144–214)

```
useEffect(() => {
  let cancelled = false
  const load = async () => {
    try {
      const res = await fetch(`/api/transcript/${id}`)
      if (res.ok) { const data = await res.json(); if (!cancelled) {
setSegments(data.segments); setTitle(data.title ?? ''); setLoading(false) } return }
    } catch {}
    if (readLocal()) { if (!cancelled) setLoading(false); return }
    if (!cancelled) { setNotFound(true); setLoading(false) }
  }
  load(); return () => { cancelled = true }
}, [id])

const active = lang !== 'en' && translations[lang] ? translations[lang] : { title,
segments }
const displayTitle = active.title; const displaySegments = active.segments
```

Even though Block 2 may have shown a local copy already, this effect fetches the authoritative version from the database (so a shared link on a fresh device works, and so the copy is current). The database is tried first; the local copy is the fallback; if both fail, `notFound` is set. The `cancelled` flag is the standard guard against a "stale response": if the effect re-runs or the component unmounts before the fetch returns, `cancelled` is set in the cleanup and the late response is ignored, preventing an out-of-order update. Below the effect, `active` chooses what to display — the cached translation when a non-English language is selected and available, otherwise the original — and `displayTitle` / `displaySegments` are what the render uses.

Block 7 — translation on demand (lines 200–249)

```
useEffect(() => {
  if (lang === 'en' || segments.length === 0 || translations[lang]) return
  // try localStorage cache first; if a valid cached translation exists, use it and return
  let cancelled = false; setTranslating(true); setTranslateError('')
  ;(async () => {
    try {
      const res = await fetch('/api/translate', { method: 'POST', headers: {'Content-
Type':'application/json'},
        body: JSON.stringify({ title, paragraphs: segments.map(s => s.text), target: lang
}) })
      const data = await res.json(); if (cancelled) return
      if (res.ok && Array.isArray(data.paragraphs) && data.paragraphs.length ===
segments.length) {
        const entry = { title: data.title ?? title, segments: data.paragraphs.map((text,
i) => ({ text, startMs: segments[i].startMs })) }
        setTranslations(t => ({ ...t, [lang]: entry }))
        localStorage.setItem(`translation-${id}-${lang}`, JSON.stringify(entry))
      } else setTranslateError(data.error ?? 'Could not translate.')
    } catch { if (!cancelled) setTranslateError('Could not translate.') }
    finally { if (!cancelled) setTranslating(false) }
  })()
  return () => { cancelled = true }
}, [lang, id, segments, title, translations])
```

Runs only when a non-English language is selected and not already cached. It checks `localStorage` for a saved translation first; failing that, it calls `/api/translate` (Chapter 7.3). The response is accepted only if the paragraph count matches the source — the alignment guarantee restated on the client — and is stored both in

memory (`translations`) and in `localStorage` , so future switches are instant. The `;(async () => { … })()` is an immediately-invoked async function: `useEffect` 's callback itself cannot be `async` , so an async function is defined and called on the spot. The leading semicolon prevents a subtle parsing hazard with the preceding line. `finally` clears the "translating" flag whether the call succeeded or failed.

Block 8 — copy and download (lines 251–290)

```
const buildExportText = () => displaySegments.map(s => s.text).join('\n\n')

const handleCopy = async () => { await navigator.clipboard.writeText(buildExportText());
setCopied(true); setTimeout(() => setCopied(false), 2000) }

const handleDownloadTxt = () => {
  const blob = new Blob([buildExportText()], { type: 'text/plain' })
  const url = URL.createObjectURL(blob)
  const a = document.createElement('a'); a.href = url; a.download = 'transcript.txt';
a.click()
  URL.revokeObjectURL(url); setDownloadOpen(false)
}

const handleDownloadPdf = () => { setDownloadOpen(false); setTimeout(() => window.print(),
50) }
```

`buildExportText` joins the (possibly translated) paragraphs with blank lines. `handleCopy` writes to the clipboard and flips a `copied` flag for two seconds to show feedback. `handleDownloadTxt` is the standard browser file-download trick: wrap the text in a `Blob` , make a temporary object URL for it, create an invisible `<a download>` , click it programmatically, then revoke the URL to free memory. `handleDownloadPdf` simply calls `window.print()` ; the actual PDF layout is a hidden two-column block in the render (styled with `@media print`) that uses the chosen reading font, so the browser's own print-to-PDF preserves the typeface.

Block 9 — the render (lines ~292–505)

The return statement assembles everything: the header (wordmark, Feed/Library buttons, the Download menu, the Aa settings button that opens `ReadingSettingsMenu`); the title with the measured underline bars drawn over it and dark-theme-aware shadows; a translating/error line; the `TranscriptReader` passed the display segments and the resolved reading styles from `prefs` (theme, font family, size, spacing, width — looked up in the imported catalogs); the delete button at the end; and the hidden print-only two-column document. The page's job is coordination: it owns *which* transcript, in *which* language, with *which* settings, and delegates the actual text interaction to the reader. Everything you read in Chapter 18 receives its inputs here.

> **The shape of a coordinator page.** Notice how little of this file is markup and how much is effects: load, translate, save progress, restore progress, measure the title. A page in this codebase is mostly wiring between storage, the network, and one or two presentational components. That is the pattern to expect when you open any other `page.tsx` .

20 · Trace-it-yourself exercises

These are reading exercises, not coding tasks. For each, follow the path through the real files and write your answer in a sentence or two before checking Chapter 21. They are ordered from concrete to conceptual. Doing them is what turns "I read it" into "I understand it."

Set A — follow the data

1. **From paste to paragraph.** A user pastes `https://youtu.be/abc123` and clicks Transcribe. List, in order, every file that runs and what each contributes, ending at the first paragraph appearing on screen. (Hint: start in `page.tsx`, end in `TranscriptReader.tsx`.)
2. **The cache hit.** The same user, an hour later, pastes the same link. Which steps from exercise 1 are skipped, and at exactly which line is the decision made?
3. **A highlight's life.** A user selects the word "Palantir" in paragraph 3 and clicks Highlight. Name the function that turns the selection into a `Span`, the function that stores it, the storage key it lands under, and what makes it reappear after a page reload.
4. **Two devices.** The user opens the same transcript on their phone. Do they see the transcript? Do they see the highlight from exercise 3? Explain each answer in terms of where the two pieces of data live.

Set B — reason about the design

5. **The alignment guard.** The translate route and the transcript page both check that the translated paragraph count equals the source count. What specific corruption does this prevent, and which stored data would be damaged without it?
6. **Why defer the selection handler?** `handleMouseUp` wraps its work in `setTimeout(…, 0)`. Describe the sequence of events when a user clicks an existing highlight, and explain what would break if the timeout were removed.
7. **The minimum-visible time.** The loading page computes `MIN_VISIBLE_MS` from animation constants. Trace what the user sees when the transcript returns from cache in 50 milliseconds, and explain why the constant is derived rather than a fixed number like `3000`.
8. **Degrade or refuse.** For each of these failures, state whether the code degrades or refuses, and why: (a) Gemini cleanup times out; (b) the free dictionary has no entry for a French word; (c) the translation returns 41 paragraphs for a 42-paragraph transcript; (d) `localStorage` contains corrupted highlight JSON.

Set C — extend it (on paper)

9. **A new correction.** A user reports that "Andrej Karpathy" is transcribed as "Andre Carpathy." Which single file and which part of it would you change, and would already-saved transcripts be fixed? Why or why not?
10. **Sync highlights across devices.** Sketch the smallest change that would make highlights follow a user to a second device. Which existing file's function signatures would you try to preserve, and why does preserving them limit the blast radius?

21 · Answers and discussion

Check your traces against these. Where your answer differs, re-read the cited block — the goal is that the reasoning, not just the conclusion, matches.

Set A

1 — From paste to paragraph. `page.tsx` validates the URL, stores it in `sessionStorage`, navigates to `/transcribing`. `transcribing/page.tsx` reads it and POSTs to `/api/transcribe`. `api/transcribe/route.ts` extracts the id, misses the cache, calls Supadata (captions), groups paragraphs, calls `cleanup.ts` → Gemini, fetches title/date, calls `db.ts` `saveTranscript`, returns segments. Back in `transcribing/page.tsx` the result is written to `localStorage` and it navigates to `/transcript/<id>`. `transcript/[id]/page.tsx` reads the local copy synchronously (Block 2) and renders `TranscriptReader.tsx`, whose `renderParagraph` (Block 9) produces the first paragraph's nodes.

2 — The cache hit. Supadata, the paragraph grouping, Gemini cleanup, and the metadata fetch are all skipped. The decision is in `api/transcribe/route.ts` immediately after id extraction: `const cached = await getTranscript(videoId); if (cached) return …`. The loading page still enforces its minimum visible time, so the animation still plays.

3 — A highlight's life. `handleMouseUp` (Block 5) builds the `Span`; clicking Highlight calls `addHighlights` → `mergeSpan` → `persist` (Blocks 7, 4), which calls `saveHighlights`. The key is `highlights-<videoId>-en`. On reload, the effect in Block 4 calls `loadHighlights` for that key and `renderParagraph` re-wraps the range in a `<mark>`.

4 — Two devices. The transcript appears, because it lives in Postgres and is fetched by `api/transcript/[id]` (Block 6). The highlight does not, because it lives only in the first device's `localStorage` under a per-browser key. This is the two-place persistence trade of Chapter 11.3 made concrete.

Set B

5 — The alignment guard. It prevents a paragraph mismatch that would shift every subsequent paragraph's text relative to its highlights. Highlights are stored as character offsets into a specific paragraph index; if translation merged or dropped a paragraph, offsets would point at the wrong words. Refusing keeps stored highlight data valid.

6 — Why defer. On release, the browser may fire both the `<mark>`'s `onClick` (open the note pop-up) and the container's `onMouseUp` (open the actions pop-up). Deferring `handleMouseUp` by a tick lets the click's handler run first; without the timeout, clicking an existing highlight could immediately be overridden by the actions pop-up, and the note pop-up would never appear.

7 — Minimum-visible time. The transcript is ready in 50ms, but `waitForMinimum` holds until `MIN_VISIBLE_MS`, so the user watches the full wave fill once, then transitions — no flash. It is derived from `WAVE_ROWS`, `ROW_APPEAR_STEP`, etc., so that if the animation is retuned the minimum stays in sync automatically; a hardcoded 3000 would silently become wrong the moment a constant changed.

8 — Degrade or refuse. (a) Degrade — fall back to heuristic paragraphs. (b) Degrade — fall back to the Gemini definer. (c) Refuse — return an error, because misalignment would corrupt highlights. (d) Degrade — treat as no highlights rather than crash. The rule (Chapter 11.5): refuse only when a wrong result would mislead or corrupt.

Set C

9 — A new correction. Add the mapping to the known-mishearings list in the `SYSTEM_PROMPT` in `src/lib/cleanup.ts`. Already-saved transcripts are *not* fixed, because they are cached in Postgres and cleanup only runs on first transcription; fixing them requires deleting and re-transcribing (or a future re-clean action). This is the staleness cost of permanent caching from Chapter 11.4.

10 — Sync highlights across devices. Introduce accounts, then reimplement `highlights.ts`'s functions (`loadHighlights`, `saveHighlights`, `countHighlights`, `loadProgress`, `saveProgress`) to read and write per-user database rows instead of `localStorage` — keeping the same names and signatures. Because every caller (the reader, the library) goes through those functions, preserving the signatures means the callers do not change; the migration is contained to one file plus new routes and a table. That containment is exactly the isolation Chapter 14 relies on.

End of the expanded edition. You have now read the project's two hardest files line by line, traced its main paths yourself, and argued its design and limits. With the source open beside these two volumes, there is no part of this codebase that should remain opaque — and the exercises in Chapter 20 are worth repeating from memory a week from now to confirm it stuck.